

Supplementary material

VCF-RDFizer: From a VCF File to Explicit, Verifiable, and Governable Semantic Genomic Data

Elias Crum, Bart Buelens, Ruben Taelman, Gökhan Ertaylan

This supplement specifies the inputs, implementation, and checks supporting the three evaluation questions in the main text. The linked experiment tests prerequisites for the motivating applications, not three complete clinical or governance systems. Completed comparisons, intended refusals, incomplete checks, and unresolved validation failures are reported separately.

Contents

S1 Documented capabilities of related tools	3
S2 Shared-input converter comparison	4
S2.1 Converters, builds, and options	5
S2.2 Questions and ports	6
S2.3 Results	6
S3 Representation and implementation details	7
S3.1 Model and sample profiles	7
S3.2 Conversion and incremental representation construction	8
S4 Validation specification, scope, and sensitivity	10
S4.1 Queries, normalization, and engines	10
S4.2 Completed comparisons and the consumer WGS validation run	11
S4.3 Fault sensitivity and oracle limitations	12
S5 Linked genomic workflow: protocol, policies, and detailed results	12
S5.1 Scope, linking, and policy evaluation	12
S5.2 Application question, comparison workflows, and evaluation arms	14
S5.3 Linking results, matches, and rule changes	15
S5.4 Stage costs	17
S6 Reproducibility and execution	18
S6.1 The base campaign	18
S6.2 Builds and dependencies	20
S6.3 Hosts, measurements, and execution status	21
S7 Input and version coverage	22
S8 Defects, corrections, and remaining gaps	23

S9 Conversion, storage, and retrieval measurements	25
S9.1 Record and sample scaling	25
S9.2 Storage mode and optional artifacts	25
S9.3 Query workloads and physical access paths	26
S9.4 Setup amortization	28
S10 Configuration coverage, operating modes, and memory feasibility	29

S1 Documented capabilities of related tools

Table S1 expands Table 1 of the main text with the evidence behind each mark. Each mark records what a tool’s public documentation states: its repository and manual, the cited paper, and package registries, inspected in September 2026 and re-checked against those sources in October 2026. A mark is not a measurement on a shared input, and an open mark means that a capability was not documented, not that it is impossible. Where a tool was superseded, the version that reads VCF was assessed. The columns are defined as follows; a partial mark applies the qualification given after each definition.

RDF model The output uses a documented vocabulary. Partial: terms shown only by example, or a vocabulary that is no longer published.

Header + genotype VCF header metadata is kept in the RDF together with per-sample genotype and FORMAT data. Partial: only one of the two, or both only through user-written templates.

Editable mappings Users can change how VCF maps to RDF without changing the converter. Partial: configuration or plug-in hooks over a fixed mapping, or mappings that produce only part of the output.

CLI Documented command-line use suitable for pipelines. Partial: library, browser, or user-interface use only.

Reproducible workflow A documented VCF-to-RDF route that a third party can rerun. Partial: manual installation of several components, an unpublished package, an offline service, or an unmaintained runtime.

Compressed RDF The tool writes compressed RDF itself. Partial: only through a bundled wrapper.

Public release A versioned package, executable, or container. Partial: source code only.

Table S1. Documented capabilities of VCF-to-RDF converters and related workflows, with the evidence for each mark. ● supported and documented; ◐ partial or limited; ○ not found in public documentation. Sources are the cited papers and the repositories, manuals, and registries named in each note.

Tool or workflow	Representation			Operation			
	RDF model	Header + genotype	Editable mappings	CLI	Reprod. workflow	Compr. RDF	Public release
<i>Dedicated VCF-to-RDF converters</i>							
VCF-RDFizer (this study)	●	●	◐	●	●	●	●
Writes the public VCF Core vocabulary, keeping the header, records, alleles, and samples (expanded or condensed). Mappings are partial: RML maps the record backbone, nine triples per record and 5.3% of the HG005 graph, while samples, INFO, QUAL, and the header come from emitters configured by options rather than mappings (Section S3.2). Pinned Docker image; HDT and COTTAS output; released on PyPI, Docker Hub, and conda-forge.							
TogoVar VCF2RDF [1]	◐	○	◐	●	●	○	●
Rust command-line tool (https://github.com/togovar/vcf2rdf) with <code>generate</code> , <code>convert</code> , and <code>compress</code> . Its documentation calls the RDF model very simple, shown through a configuration template. TogoVar converts allele frequencies only, for privacy, so no sample or header data. A YAML configuration selects the base IRI, namespaces, and INFO keys over fixed mapping rules. <code>compress</code> writes BGZF-compressed VCF, not RDF. Docker image and release binaries.							
JVarkit VcfToRdf [2]	◐	◐	○	●	●	○	●
<code>java -jar jvarkit.jar vcf2rdf</code> (documentation at https://jvarkit.readthedocs.io). A project-specific <code>vcf:</code> namespace is shown only by example. Output includes chromosomes, filters, samples, and genotypes (GT, DP, GQ, PL), but not INFO or FORMAT header descriptions, and the documented default of <code>--hide</code> includes GT. The only options are <code>--hide</code> and <code>-o</code> . Released as a jar and on Bioconda.							

Table S1, continued

Tool or workflow	RDF model	Header + genotype	Editable mappings	CLI	Reprod. workflow	Compr. RDF	Public release
SPARQLing Genomics [3, 4] vcf2rdf, documented in the v0.99.11 manual. Uses FALDO, Dublin Core, PROV, and its own terms, with field predicates generated from the VCF header; the manual’s namespaces are inconsistent. Every header-described field becomes triples, with per-sample calls carrying INFO and FORMAT values (only calls with an ALT allele). The mapping is fixed in code. Compression only through the <code>--compress</code> option of the bundled <code>folder2rdf</code> wrapper. Release tarball, Docker image, and Guix environment; the GitHub repository was archived in 2021 when development moved to GitLab.	●	●	○	●	●	●	●
VCF2RDF (web) [5] JavaScript for browsers and Node.js (https://github.com/ib1/VCFr); no command line is documented. Terms are minted under vcf2rdf.org/app/ , a host that is now offline, with no published schema. The paper describes header classes but not per-sample genotypes. Source code only; the web service is offline.	●	●	○	●	●	○	●
BioHackathon vcf2rdf [6, 7] Built at BioHackathon 2013 (https://github.com/fstrozzi/bioruby-vcf2rdf), with a <code>vcf2rdf</code> executable. FALDO positions with an ad hoc allele namespace. Per-sample genotypes and allele coverage are written; header metadata is not mentioned. Turtle output. The Ruby gem the README installs is not published, so only the source is available.	●	●	○	●	●	○	●
<i>General genomic-format tools</i>							
BioInterchange [8, 9] Assessed as BioInterchange 2 (v2.0.5, https://github.com/BioInterchange/BioInterchangeC); version 1, conceived at BioHackathon 2012, does not read VCF. Converts VCF 4.2 to JSON-LD in the Genomic Feature and Variation Ontology (GFVO) [10], with one metadata object per file and per-sample genotypes on each feature. Output can be changed through Python plug-in hooks and user annotations, not mapping files. RDF serializations come from JSON-LD tools; compression was applied outside the tool. Homebrew package and installer.	●	●	●	●	●	○	●
bio-vcf [11] <code>bio-vcf --rdf</code> and ERB templates such as <code>vcf2rdf.erb</code> (https://github.com/vcflib/bio-vcf) define the output, with ad hoc prefixes rather than a documented vocabulary. Header metadata and sample data are both available to templates but are combined only by the user. Released as a Ruby gem and on Debian and Bioconda.	○	●	●	●	●	○	●
ruby-rdf/rdf-vcf [12] <code>vcf2rdf file.vcf.gz</code> (https://github.com/ruby-rdf/rdf-vcf), requiring JRuby 9 and a tabix index; unmaintained since its 0.1.2 gem in September 2015. The documentation names no vocabulary and does not mention header, INFO, FORMAT, or genotype handling. Uncompressed N-Triples.	○	○	○	●	●	○	●
<i>Knowledge-graph construction pipelines</i>							
VariantKG [13] Annotates VCF with SnpEff, converts it with SPARQLing Genomics’ <code>vcf2rdf</code> (and so inherits its header and genotype coverage), then extends that vocabulary with its own ontology, including CADD scores, and links patient metadata through named graphs; no ontology file is published. Operated through a Gradio interface after manual installation of SnpEff, <code>vcf2rdf</code> , and Blazegraph. Source code only (https://github.com/MU-Data-Science/GAF).	●	●	○	●	●	○	●
AgroLD VCF2RDF [14] Named in [14] among the AgroLD conversion pipelines (GFF2RDF, GAF2RDF, VCF2RDF) at https://github.com/SouthGreenPlatform/AgroLD_ETL . AgroLD itself is ontology-based, but no VCF vocabulary, usage, or output is documented. Source code only.	●	○	○	○	○	○	●
<i>Virtual semantic access</i>							
Semantic Beacons [15] Maps Beacon API responses, not VCF files, to RDF on the fly with RML over JSONPath, served as Triple Pattern Fragments and queried with Comunica. Its SO, FALDO, and GENO schema covers variant positions and alleles, which the authors describe as partial coverage of the Beacon specification; Beacon responses carry no header or per-sample genotypes. Implementation details are deferred to https://github.com/SemanticBeacon/SemanticBeacon .	●	○	●	○	●	○	●

S2 Shared-input converter comparison

Table S1 records what each converter documents. This section reports what four of them write when they convert the same inputs as VCF-RDFizer, measured with the paper’s existing measurements: the validator’s content questions Q1–Q8 against the oracle, conversion wall time and peak resident memory, triples per record, and gzipped N-Triples size (main Figure 4). The run is experiment 18 of `vcf-rdfizer-testing` at commit `e3743590`, on `vcf-bench-1` on 8 October 2026, with three replicates of every conversion; its records are

archived in `benchmark-results/vcf-bench-1/18_converter_comparison/`. The whole experiment runs from one Docker Compose file, so the host needs only Docker: `CONV_DATA=... benchmarks/converters/run.sh`.

S2.1 Converters, builds, and options

Each converter ran in its upstream release image wherever upstream publishes one, and with the options that make its output most complete rather than with its defaults (Table S2). Images built for the experiment start from a Debian base pinned by digest and take their packages from `snapshot.debian.org` at a fixed date; every downloaded artifact was checked against a pinned SHA-256, and the run records the resolved Compose file and every image ID. Converter containers ran without network access, as the calling user. A static BusyBox `time -v`, mounted read-only into each container, measured wall time and peak resident set size, the quantities VCF-RDFizer’s own run summary reports for its stages; VCF-RDFizer’s wall time spans all its stages, including starting a container for each. Apache Jena `riot` 6.2.0 streamed each native output to N-Triples, which were counted, gzipped, and indexed with the QLever build in the VCF-RDFizer image.

Table S2. Converters in the shared-input comparison: the version run, how it was obtained, and the options used.

Converter	Version	Obtained as	Options; output
VCF-RDFizer	v3.3.1 (b25fb7b)	Release image <code>sha256:3ad71b1a...2993</code> , driven by the release’s command-line tool	Expanded samples, space-optimized storage, N-Triples only; gzip-framed N-Triples
JVarkit <code>vcf2rdf</code> [2]	Commit <code>fc2356a3</code>	Built from source, with its git submodule; no jar is published, and the release tag was moved after publication, so the commit is the pin. The SHA-256 of every library jar the build fetched is recorded	<code>--hide ''</code> (the default hides ALT, FILTER, and genotypes); Turtle
TogoVar <code>vcf2rdf</code> [1]	v1.0.0-beta.12 (f17835a6)	Upstream Docker Hub image, by digest	<code>generate config</code> for the input’s assembly, then <code>convert --no-normalize</code> , which keeps POS, REF, and ALT as written; Turtle
SPARQLing Genomics <code>vcf2rdf</code> [4]	0.99.11 (1859f6f8)	Upstream release image (a Guix pack), loaded from its release tarball after a checksum check	Defaults, which already write INFO, FORMAT, and the header; N-Triples
BioInterchange [9]	2.0.5 (49d57012)	The brewsci Homebrew bottle, still served by its registry and addressed by its SHA-256. The source build no longer works, because a dependency’s repository has been deleted	Defaults; line-delimited JSON-LD, whose contexts now return HTTP 404 and were inlined from the pinned commit

The other entries in Table S1 were not run. The VCF2RDF web service is offline and documents no command line; the BioHackathon `vcf2rdf` gem was never published; `rdf-vcf` has been unmaintained since 2015 and names no vocabulary; `bio-vcf` writes RDF only through user templates without a documented vocabulary; VariantKG converts with SPARQLing Genomics’ `vcf2rdf`; AgrolD documents no VCF conversion; and Semantic Beacons maps Beacon responses rather than VCF files.

S2.2 Questions and ports

Every graph was compared by the same script, inside the VCF-RDFizer image, with that image’s validator: the same oracle over the source VCF, QLever build, result normalization, and comparator. Only the SPARQL differs: each converter’s Q1–Q8 are ported to its vocabulary, and VCF-RDFizer’s are its validator’s own queries. A question a graph cannot answer has no port; its reason, with evidence from the converter’s source, is recorded instead and reported as not represented, and the script refuses to run when a question has neither. Q9–Q13 were excluded: Q9 and Q10 count VCF Core predicates and classes, and Q11–Q13 hash VCF-RDFizer’s record IRIs.

Ports read only what a graph states. Where a converter writes two VCF values the same way, the port reads that state as the value the converter’s code derives it from, and the comparison shows whether the conflation costs anything on these inputs: Jvarkit writes no FILTER triple for either PASS or ‘.’ (read as PASS), writes no ploidy (haploid calls read as homozygous), and types neither a no-call nor a record without GT; TogoVar replaces CHROM with a reference IRI from its configuration, which the Q1 port maps back through the same configuration; SPARQLing Genomics writes no record node, so its calls are regrouped through the record index in their IRIs, as its manual documents, and it writes only the first ALT allele; and BioInterchange writes a missing allele as REF, writes ‘*’ as an empty string, and strips the VCFv prefix from `##fileformat`. TogoVar and SPARQLing Genomics lose the order of multiple FILTER codes.

S2.3 Results

Every conversion and comparison completed (52 recorded steps, all with exit status 0). VCF-RDFizer answered Q1–Q8 as the oracle did on both inputs; the other graphs’ differences come from what they hold:

- **Jvarkit.** Its record IRI is `urn:variant/CHROM:POS:REF`, so the five pairs of 1000 Genomes lines that share CHROM, POS, and REF (for example `20:283049 C>CGAT` and `C>T`, both with ID ‘.’) merge: 9,995 records instead of 10,000, five more multiallelic records, and every later count shifted. On the HG005 slice it answered Q1–Q6 as the oracle did.
- **TogoVar.** It drops alleles outside `[ACGTURYKMSWBDHVN]`: the 13 symbolic records of the 1000 Genomes slice disappear, and `chr1:1663386 C>*,T` on the HG005 slice becomes an SNV (Q2, Q3). The same five pairs merge, because TogoVar writes no record node. It writes no samples or header lines, by design.
- **SPARQLing Genomics.** A call is written only when it carries an alternate allele, so the 1000 Genomes graph holds the 1,401 of 10,000 records with a carrier, no homozygous-reference calls, and allele numbers over carriers only (Q6); multiallelic records are recognized only through an allele index of 2 or more, 19 of 70. On the HG005 slice, where every call carries an alternate allele, it answered Q1–Q7 as the oracle did; htlib adds a PASS FILTER definition the file does not declare (Q8).
- **BioInterchange.** Its header document for the HG005 slice (line 2 of 100,003) is not valid JSON, because the tool writes strings without escaping them, so the header is absent from that graph (Q7, Q8). All 100,001 record documents parse, and on the 1000 Genomes slice it answered all eight questions as the oracle did.

Replicates. Each converter’s three outputs were byte-identical, with two exceptions. VCF-RDFizer’s gzip framing differed between replicates, but its sorted triples were identical in all three replicates of both inputs; on the HG005 slice they equal those of the v3.1.0 conversion (Table S13).

Table S3. Conversion cost and output size in the shared-input comparison: median wall time and largest peak resident set size over three replicates, and the size of replicate 1’s output once streamed to N-Triples. QLever index times are for the same graphs.

Converter	Input	Wall (s)	Peak RSS (GB)	Triples	Triples per record	N-Triples.gz ÷ VCF.gz	QLever index (s)
VCF-RDFizer	HG005	91.6	1.85	17,098,746	171.0	25.5	20.8
	1000 Genomes	31.4	0.93	5,359,578	536.0	136.9	7.0
JVarkit	HG005	2.2	0.43	1,401,613	14.0	2.1	1.8
	1000 Genomes	1.4	0.30	1,020,454	102.0	18.2	1.4
TogoVar	HG005	1.3	0.01	5,728,120	57.3	15.4	6.8
	1000 Genomes	0.1	0.01	564,940	56.5	35.0	1.0
SPARQLing Genomics	HG005	6.3	0.01	2,958,769	29.6	4.8	3.3
	1000 Genomes	0.4	0.01	180,053	18.0	4.1	0.5
BioInterchange	HG005	1.5	0.02	4,559,438	45.6	17.0	6.6
	1000 Genomes	0.4	0.01	2,031,215	203.1	154.3	3.2

BioInterchange’s replicates differed only in their **Context** and **Summary** documents, which record run paths and timestamps.

IRI warnings. riot parsed every output, warning about three converters’ IRIs: VCF-RDFizer’s record IRIs, `file://<name>.vcf#record/n`, place the file name where a URL’s host would be (an “unwise” IRI; 23.6 million warnings on the HG005 graph); SPARQLing Genomics’ `origin://<md5>@<k>` uses deprecated user information (3.1 million); and JVarkit’s `urn:variant/...` has an invalid URN namespace (0.6 million). None prevented parsing or indexing; TogoVar’s and BioInterchange’s outputs drew no warnings.

S3 Representation and implementation details

S3.1 Model and sample profiles

VCF-RDFizer uses the VCF Core vocabulary (`vcfc:`, <https://w3id.org/vcf-core/vocab#>) to retain file and header assertions, variant records, and sample/genotype information [16]. The model aligns with FALDO, the Sequence Ontology, HERO, and features of GA4GH VRS [17, 18, 19, 20]. Its classes, properties, domains, and ranges describe the representation; SHACL supplies value and cardinality constraints rather than OWL restrictions. Figure S1 shows the emitted backbone, and Table S4 distinguishes implemented capabilities from assessed coverage. The documentation-based converter comparison is now Table 1 of the main text.

In expanded output, sample calls and their **FORMAT** values are addressable resources; condensed output stores the corresponding values in sample-ordered vectors. Both retain the same file, header, and record context. For V records, S samples, and F **FORMAT** keys in a uniform layout, expanded output adds VS sample calls and VSF **FORMAT**-value resources, while condensed output adds V matrices and VF vectors. Excluding shared structure and assuming bounded ploidy and value arity, their sample-layer triple counts scale as

$$\begin{aligned} T_{\text{expanded}} &= \mathcal{O}(S + VSF), \\ T_{\text{condensed}} &= \mathcal{O}(S + VF). \end{aligned} \tag{1}$$

Condensed output still contains all VSF values inside literals, so its bytes grow with sample count even when its repeated triple structure does not. Deterministic IRIs identify observations by source file and record position; linking supplies cross-file identity separately. GA4GH VRS computed identifiers could complement the evaluated SPDI route, but were not implemented in this study [20].

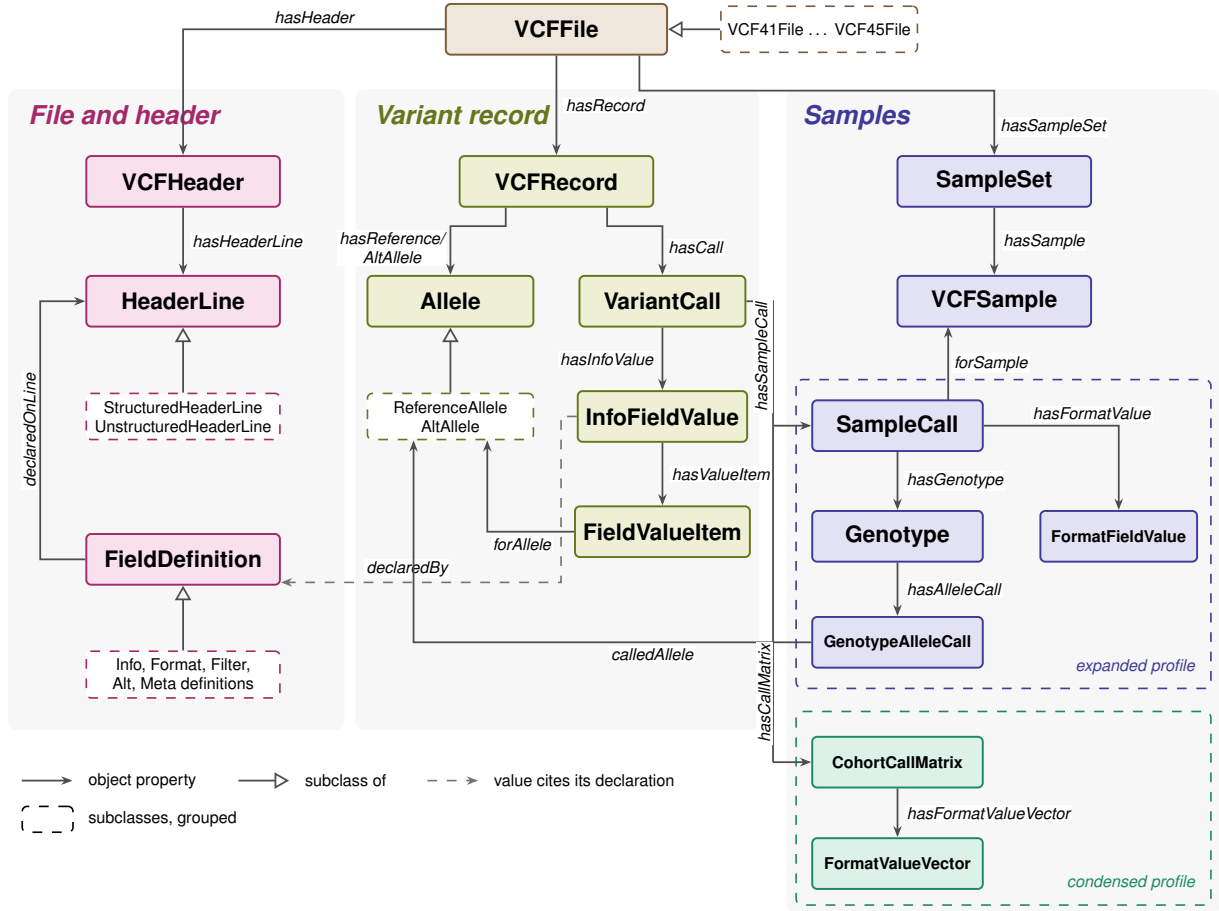


Figure S1. Simplified VCF Core model emitted by the evaluated converter. All terms use the `vcfc:` prefix. A versioned file resource connects header declarations, records, and sample information; values retain links to their interpretation context. Expanded output exposes sample calls, genotypes, and allele calls, while condensed output uses sample-ordered vectors. Structural-variant, phasing, local-allele, and value-typing classes are omitted; the complete vocabulary is documented at <https://ecrum19.github.io/vcf-core-vocabulary/>.

S3.2 Conversion and incremental representation construction

The conversion first derives record, header-line, and file-metadata TSV logical sources, because the implemented RML path does not read VCF directly. Turtle-encoded RML mappings executed by RMLStreamer produce the record backbone [21, 22]. Direct emitters add sample structures and content requiring per-value handling, including typed INFO and FORMAT values, QUAL, and header attributes. RML contributes nine triples per record, 5.3% of the HG005 graph; the emitters supply the other 94.7%. For example, the implemented path delegates datatype selection to emitters so that a missing QUAL becomes `vcfc:Null` rather than a decimal. These figures describe this hybrid implementation, not an inherent limitation of every RML-based architecture.

Each part is appended to plain N-Triples or as one member of a concatenated-gzip N-Triples stream, then deleted (main Figure 1). Standard gzip readers expose the members as one stream. Storage, RDF compression, representation construction, and artifact packaging are separate options: `--rdf-storage-mode`, `--rdf-compression`, `--representations`, and `--artifact-compression`. HDT and COTTAS can therefore be retained independently of additional transport packages.

Chunking and merging. Representation builders read the aggregate incrementally, respecting complete N-Triples lines and keeping at most one uncompressed chunk at a time. Each selected

Table S4. Implemented capabilities and assessed scope. Each status gives release support, then the completed evaluation in *italics*; unprefixed section references are to the main text.

Capability	Status	Notes and limits
Core fields (CHROM, POS, ID, REF, ALT, FILTER)	Supported (v3.1.0) <i>Validated</i>	Q1–Q4 equal in completed comparisons and in the consumer WGS validation run (Section S4.2)
QUAL	Supported <i>Q11 record digest</i>	Typed literal for every record; a missing QUAL is <code>vcfc:Null</code>
INFO	Supported; structured by default <i>Q11, Q12 digests</i>	Typed value per record and key, per-allele items for Number=A/R/G, plus the complete <code>infoRaw</code> string
FORMAT, sample, and genotype values	Supported <i>Q5–Q6 exact; Q13 digest</i>	Half-calls, haploid calls, and ploidy 4/3 survive conversion (Table S15); called allele, sample index, and phasing require the deeper shapes for the tested mutations
Header metadata	Supported <i>Census queries and shapes</i>	Header attribute–value agreement requires the deeper shapes, not merely the default profile
Multi-sample conversion	Supported <i>Measured, 1–2,504 samples</i>	Condensed adds 5 triples per sample; expanded adds 25 per sample per record (627M triples for 10,000 records at 2,504 samples), so the whole cohort was converted in neither (Section 4.3.1)
Structural and symbolic alleles	Lexically supported <i>HGSVC2 converted; Q2 passes on symbolic and breakend fixtures</i>	ALT remains the source lexical field; allele-value mutations are caught only by the full shape profiles
Queryable HDT, COTTAS	Supported <i>Measured</i>	118 HDT and 34 COTTAS artifacts, all decode checks passed; COTTAS smallest throughout (Section 4.3.2)
Gzip/Brotli packaging	Supported <i>Covering set (Section S10)</i>	Packages must be unpacked before querying; no codec comparison
Policy annotations	Plug-in v0.2.0 (v3.3.0) <i>Real VCFs, up to a complete single-sample VCF</i>	ODRL on files and declared graph selections, DUO purposes; every view checked against a VCF-side oracle; simulated consents, single-sample files; governed release, not anonymization (Section 4.2)
External linking	SPDI, Ensembl-gene, and MyVariant linkers (v3.3.0) <i>Real VCFs</i>	SPDI joins eligible alleles under the stated reference and normalization assumptions; tier 3 confirmed 92–93% of rsID rewrites (Section S5)

builder consumes that chunk before it is deleted. HDT parts are produced by `rdf2hdt` and merged pairwise in a balanced tree with `hdtc create`, using the logical merge of HDTCat under a fixed memory budget [23, 24, 25, 26]. Unpaired parts continue to the next round, merge scratch space is removed after use, and only the final sibling `.hdt.index.v1-1` index is retained. COTTAS parts are built with `pycottas` and merged in one streaming k -way pass, avoiding a global in-memory hash table or full-size temporary spill [27]. Version 3.1.0 writes an `spo` index; v3.2.0 adds other index orders. Each operation uses an isolated temporary directory to avoid reusing a DuckDB scratch database. Parts, indexes, and builder scratch space still require disk, but a full decompressed duplicate of the aggregate is not created.

Default chunk sizes are a 512 MiB target, 128 MiB minimum, and 1 GiB maximum, controlled by `--chunk-target-bytes`, `--chunk-min-bytes`, and `--chunk-max-bytes`. Six memory-feasibility cells instead used 128, 64, and 256 MiB, respectively, to trade additional chunks for lower peak workspace. Construction, merging, final indexing, validation, and packaging are timed separately. Before raw RDF is removed, each artifact is decoded with its native reader and its triple count compared with the source; a mismatch fails the run. This checks decoding and counts, not complete RDF-term equality. The native COTTAS query path uses a DuckDB-backed Co-

munica engine over its Parquet tables [28]. QLever instead decodes an input artifact and builds its own index, so its query timings do not measure native compressed-format access.

S4 Validation specification, scope, and sensitivity

S4.1 Queries, normalization, and engines

The validation layer compares thirteen inspection queries with answers derived from the source VCF using `cyvcf2` and exact `FILTER` text from `bcftools query` [29, 30, 31]. Table S5 defines the fields and summaries compared. Fourteen preflight queries first inspect blank nodes, empty values, missing tokens, POS typing, core-property cardinality, sample representation, sample/genotype inventories, and triple counts.

Table S5. The thirteen source-comparison queries. Each normalized SPARQL result must equal its parser-derived counterpart; Q4 uses exact `bcftools FILTER` text. Ti/Tv, call rate, and allele frequency are derived after their counts agree. Q11–Q13 compare hash-bucket summaries; Q9, Q10, Q12, and Q13 depend on the selected representations.

Query	Result compared, fields, and scope
<i>Record level: every input</i>	
Q1 Record density	Records per contig and 1-Mb window (<code>CHROM</code> , <code>POS</code>); checks coordinates and integer positions
Q2 Allele shapes	Records per class: SNV, multiallelic, insertion, deletion, substitution, symbolic/breakend, no ALT (<code>REF</code> , <code>ALT</code>)
Q3 Ti/Tv	Transitions, transversions, and biallelic SNVs; single-base A/C/G/T alleles only
Q4 FILTER values	Records per exact <code>FILTER</code> string: <code>PASS</code> , <code>.</code> (not applied), or failed codes, combined codes kept whole
<i>Genotype level: inputs with sample columns and GT</i>	
Q5 Genotypes	Calls per sample and genotype class, including missing, haploid, and other-ploidy calls; phased and unphased alike
Q6 Allele counts	Sites per recomputed (<code>AC</code> , <code>AN</code>) pair; single-ALT sites with complete 0/1 haploid or diploid calls
<i>Metadata and graph structure</i>	
Q7 File metadata	Exact <code>fileformat</code> , <code>reference</code> , and <code>source</code> strings, with markers for absent declarations
Q8 Header census	<code>##</code> lines per header key (<code>INFO</code> , <code>FORMAT</code> , <code>FILTER</code> , <code>contig</code> , ...)
Q9 Predicate census	Triples per predicate against the counts expected from the VCF
Q10 Class census	Resources per <code>rdf:type</code> against the expected inventory
<i>Value preservation: IRI-bound digests</i>	
Q11 Record digest	Records per bucket over each record’s IRI, <code>CHROM</code> , <code>POS</code> , <code>ID</code> , <code>REF</code> , <code>ALT</code> , <code>QUAL</code> , <code>FILTER</code> , and raw <code>INFO</code>
Q12 INFO digest	Structured <code>INFO</code> values per bucket, bound to record and key; empty in raw-only <code>INFO</code> mode
Q13 FORMAT digest	<code>FORMAT</code> values (expanded) or sample-ordered vectors (condensed) per bucket, beyond <code>GT</code>

Answers must be exactly equal after normalization: strings for Q7, and integer counts for the other queries. Contig and sample identifiers are retained, `PASS` remains distinct from `.`, and missing, extra, or differing rows are reported. Q5–Q6 require sample data with `GT`; each source is compared only with its own graph, not with inputs on another assembly. Q11–Q13 bind values to record IRIs and count them in 256 buckets using the first byte of SHA-256. Changes that preserve the bucket histogram can escape detection, so these checks extend coverage without establishing value-by-value equality.

The same SPARQL runs under QLever, Comunica over N-Triples, Comunica over HDT, and the DuckDB-backed COTTAS engine [32, 33, 34, 35, 28]. Artifact decoding permits each engine to be assessed through different input formats; `--validation-engine all` records four-engine

agreement. The default SHACL profile checks cardinality and datatypes, and two deeper profiles check uniqueness and value agreement. A pass establishes the specified summaries or enabled constraints, not call accuracy, biological interpretation, or completeness of the entire graph.

S4.2 Completed comparisons and the consumer WGS validation run

The base campaign (Section S6.2) attempted 78 validations, in its Difficult inputs (20), Covering set (23), Equivalence (16), Query cost (18), and Modes (1) families, and completed 76, yielding 984 equal applicable query comparisons and four not-applicable genotype comparisons on empty-input artifacts (Table S6). The paired checks reached 17.1M triples; the two malformed sites-only validations did not complete. The default shapes ran in 58 validations up to 0.96M triples, with no violations and durations of 0.8–396.8s. A repeated conversion produced 17,098,746 triples with identical SHA-256 and no duplicate lines; packaging round trips preserved the digest, and re-indexing left the artifact unchanged. These are repeatability checks for the tested configuration, not every later build.

Table S6. Validation results by assessment layer. Counts report completed checks; fault sensitivity applies only to the defined mutation set and enabled profiles.

Check	Passed	What it establishes
<i>Is the graph faithful to the source VCF?</i>		
N-Triples syntax (rappier)	76/76	The serialization parses
Paired queries vs cyvcf2 (Q1–Q13)	984/984 [†]	Exact equality of normalized query results
Cross-check invariants	1,144/1,144	E.g. transitions + transversions = biallelic SNVs
Cross-engine agreement	25/25	Four-engine validations; 51 other completions used one engine
Native decode and triple count	152/152	118 HDT, 34 COTTAS; source = decoded
<i>Is the pipeline reproducible?</i>		
Determinism, two runs	1/1	Identical SHA-256; 17,098,746 triples, 0 duplicate lines
Compress and decompress	1/1	Lossless through the packaging path
Index idempotence	1/1	Re-indexing does not perturb the artifact
<i>Would a fault actually be noticed?</i>		
Faults caught, queries only	96/113 (0.850)	17 faults, spanning 10 mutation classes, are invisible to the queries
with the default shape profile	96/113 (0.850)	Cardinality and datatypes catch none of the 17
with all three shape profiles	113/113 (1.000)	Uniqueness and value agreement catch all 17; both controls still pass
SHACL shapes, default profile	58/58	Cardinality and datatype constraints, 0 violations

[†]Four further comparisons were verified as not applicable and are excluded from the 984 applicable comparisons.

The separate retrieval run on the complete HG005 VCF (all 3,856,856 records) compared all thirteen answers at 657,425,805 triples without SHACL (Table S16). It is not included in the 984-comparison denominator. The consumer WGS validation run, on the first 250,000 records of NG131FQA1I (Table S14; 58,231,176 triples), is also separate and was performed twice on **vcf-bench-2**. With v3.1.0, a first attempt exhausted host memory under the default shapes (Section S8), and a rerun without shapes agreed on ten of the thirteen answers. Q9 and Q10 differed because the oracle omitted 30,910 phase sets emitted from PS, and Q11 differed because 1,998 QUAL values with trailing zeros were returned in canonical decimal form by QLever. Direct comparison found every record field equal to the source VCF text, and canonicalizing QUAL reproduced QLever’s digest buckets exactly, so all three differences lay in the validator rather than the conversion. The corrected validator, released in v3.3.1, counts phase sets, structural-variant events, confidence intervals, and gVCF reference blocks as the emitters write them, drops a decimal QUAL’s trailing zeros on both sides of the Q11 digest, and validates shape profiles whose constraints are local to a record in batches of whole records. Rerun on the same graph, all

thirteen answers agreed, and the default shapes reported no violations in 117 batches of about 500,000 triples, taking 89min with four workers while available memory stayed above 19.9 GB. Their 46 warnings are VCF 4.5’s recommended Source and Version attributes, absent from the file’s INFO declarations. Measured on this graph, a batch costs about 2.4 GB and 300 s per million triples (1.26 GB and 155 s for 0.51M triples; 4.72 GB and 607 s for 2.0M).

S4.3 Fault sensitivity and oracle limitations

The fault set contains 113 targeted corruptions, including changed coordinates, alleles, filters, file metadata, phasing, and sample indices. Query comparisons detected 96; adding default shapes did not detect any of the remaining 17. Those 17 span ten classes: allele kind, allele value, header attribute values, derived contig count, FORMAT-item-to-allele links, field-to-header links, record index, sample index, called allele, and phasing status. All three shape profiles together detected the full set, while both unmutated controls passed under both sample profiles. The deeper profiles took 33.7 and 73.2 s on approximately 2,000 triples, against 1.7 s for the default profile. Thus their detection result is limited to the constructed fault set and small graphs, not an estimated error-detection rate for arbitrary genomic RDF.

Version 3.1.0 applies the default shapes automatically up to a 512 MiB size gate and permits all profiles when explicitly requested with `--shacl-profile full`, up to 16 MiB. Different modes used different size measures; the resulting failures and corrections are recorded in Section S8. The completed default-profile mutation rerun confirms 96/113 detection, rather than leaving its sensitivity as an open experiment. Release v3.3.1 applies the default shapes at any size, in record batches, while the deeper profiles, whose SPARQL-based constraints compare records with one another, are still validated whole: on request up to 16 MiB of VCF and, by default, not above 10M triples.

The graph and parser routes read separate representations but share some assumptions. Census expectations encode the selected representation, allele counts share a counting definition, and the linked workflow shares preprocessing and its case definition. The missing-allele defect required correction of both the emitter and its census expectation, illustrating a shared error (Section S8). Direct text comparison and structural checks therefore complement the oracle rather than make it infallible. The v3.3.1 oracle adds expectations for phase sets, structural-variant events, and gVCF blocks; local-allele constructs remain an explicit coverage limit.

S5 Linked genomic workflow: protocol, policies, and detailed results

S5.1 Scope, linking, and policy evaluation

The linked workflow combines source observations with external evidence and requester-specific release rules, and so tests core prerequisites of the three motivating use cases rather than implementing any of them in full (Section 3.3 of the main text). For clinical variant interpretation and reanalysis, it tests joins between source observations and separately maintained clinical assertions and population frequencies at fixed releases; tracking interpretation changes over time and case-level reanalysis would require additional work. For pharmacogenomic clinical decision support, it tests the cross-resource linking that would carry source observations into an interpretation and clinical-context layer; medication orders, pharmacogenomic allele calling, and prescribing recommendations would require additional work. For consent-aware genomic data reuse, it evaluates proposed uses against simulated purposes, withdrawals, and selection rules; real consent adjudication and authenticated access would require additional work. Figure S2 traces the implemented path from VCF files to a permitted answer.

1. Prepare each file and add explicit links

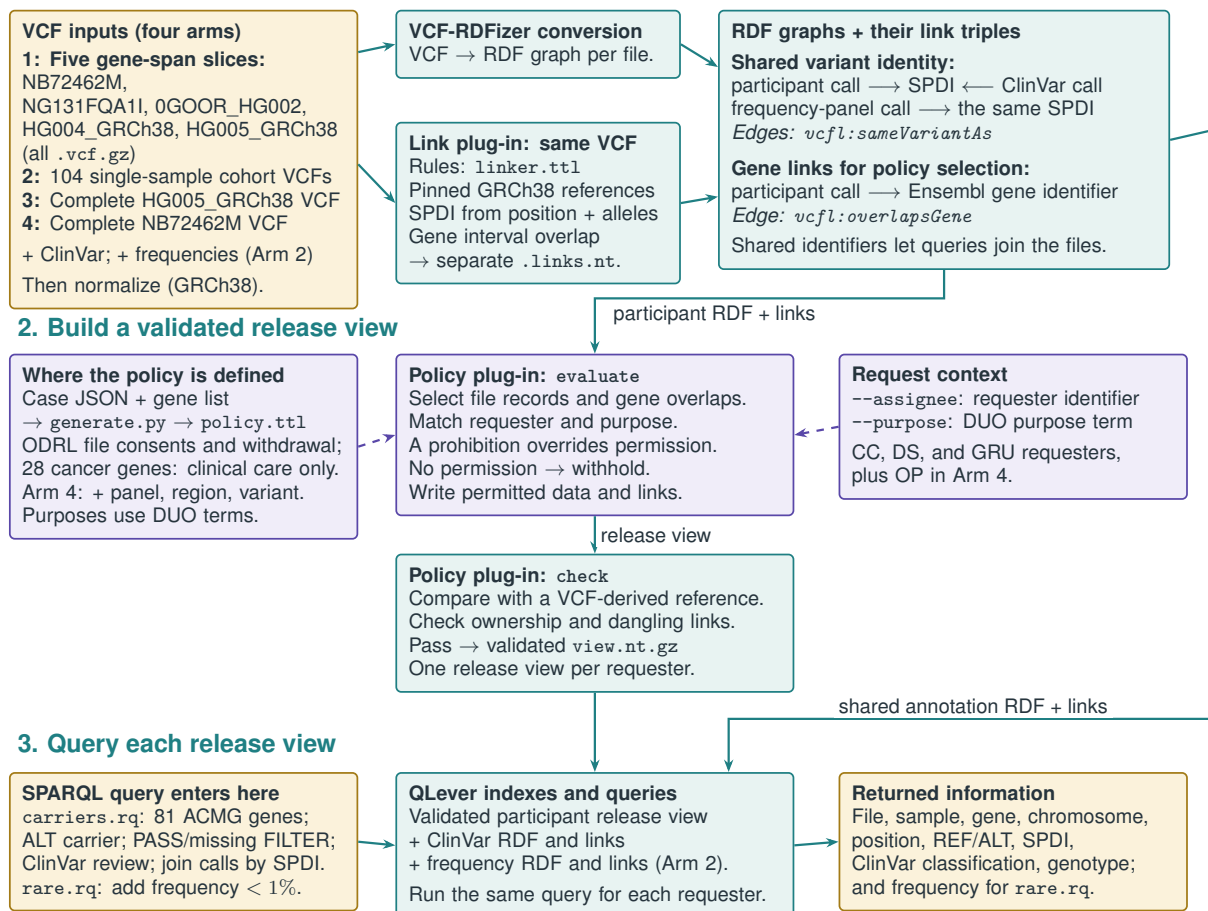


Figure S2. The implemented path from VCF files to a permitted answer. The diagram follows the use-case runner, policy generator, linker definitions, and SPARQL files. Linkers read the normalized VCFs directly, and conversion output and links are stored separately; inputs are restricted to the ACMG gene spans except in Arms 3 and 4. Policies are enforced when each release view is materialized: requester identity and purpose are evaluation inputs, while the SPARQL query holds the analysis filters. The cancer-gene policy uses Ensembl overlap links, whereas the query reports the ACMG genes named in ClinVar’s annotation. Requesters are clinical care (CC), disease-specific cardiovascular research (DS), general research use (GRU), and, in Arm 4, care by the participant’s own physician (OP).

Two plug-ins extend the VCF RDF graph without modifying the converted source representation (Section 2.3 of the main text).

Linking. *vcf-rdfizer-link* executes linkers declared in Turtle, each defining a join strategy, a reference resource, and the relationship it emits, and writes the links to a separate N-Triples linkset with a report of coverage and skipped records. Tier 1 rewrites existing tokens, such as rsIDs, into IRIs without confirming them. Tier 2 joins a digest- and assembly-pinned reference: *spdi* minimally trims explicit alleles and uses RefSeq accessions, while *ensembl-genes-grch38* links calls to Ensembl 116 genes overlapped by their REF span [36]. Contig aliases are harmonized, but minimal trimming does not establish canonical identity in every repeat context. Tier 3 uses a Python resolver to confirm identifiers through a live service, with declared request rates and budgets, digest-recorded cached responses, and no published partial linkset on failure. The *rsid-myvariant* linker uses MyVariant.info [37]. For RDF inputs, join fields are read through SPARQL from an endpoint or a temporary Oxigraph store.

Policy evaluation. `vcf-rdfizer-policy` evaluates ODRL rules over files or declared graph selections, including genomic regions, named variants, and gene-linked records [38]. Ownership rules in its VCF Core profile define which dependent resources are removed when a file or record is withheld: a withheld record’s calls, alleles, and sample calls, and a withheld file’s header. The profile applies default-deny and deny-overrides semantics; permitted purposes are drawn from a subset of DUO by default and can use other RDFS or SKOS vocabularies [39]. Version 0.2.0 evaluates against an endpoint and materializes each release view by streaming the selected N-Triples in their original order, without rewriting them. Release validation requires each release view to match the policy digest, contain no prohibited or unpermitted owned resources, have no dangling references, and return the same records as an oracle derived from the source VCF text. The dangling-reference check queries a second endpoint serving the view. An extra record is an unintended release, and a missing permitted record is over-withholding. Released genotypes retain their source IRIs and are not anonymized; obligations are recorded but not enforced.

Selector parameters are inserted as an inline **VALUES** block inside the outer query group. A trailing **VALUES** clause is joined after that group, which left the tested region selector empty and its prohibition ineffective. The placement correction is necessary for the evaluated rule behavior; it does not replace tests of each newly declared selector.

S5.2 Application question, comparison workflows, and evaluation arms

Application question. The application asks which observed variants in the fixed 81-gene ACMG SF v3.2 secondary-findings set have a corresponding ClinVar classification, which participant and gene are associated with each match, and which matches may be released to each requester [40]. ClinVar variants are included when they have at least one review star, regardless of classification. Pathogenic and likely pathogenic (P/LP) matches are reported separately; they are not treated as reportable secondary findings, because the workflow does not evaluate the additional gene-, inheritance-, and variant-specific criteria required for clinical reporting.

RDF and conventional comparison workflows. The RDF workflow links the converted VCFs to ClinVar and gene resources with the SPDI and Ensembl-gene linkers, materializes a release view for each requester against QLever, and queries each view and ClinVar with SPARQL [32]. The conventional workflow harmonizes contig names, annotates **CLNSIG**, **CLNREVSTAT**, and **GENEINFO** with bcftools, and evaluates the same case definition and release conditions in a separate Python implementation [30]. The two workflows are compared using sorted tuples defined by participant, gene, contig, position, and alleles, which must agree exactly for every requester before timings are reported. Counts therefore represent participant–variant–gene matches, not unique participants or independently adjudicated findings. Both workflows share preprocessing and the case definition; the downstream linking, policy evaluation, and query implementations are separate.

Evaluation arms and simulated release policies. Each experimental arm is a distinct dataset and policy scenario. Arm 1 contains the ACMG gene-span regions of five single-sample VCFs from three sources: two Personal Genome Project donations (NB72462M and NG131FQA1I), the HG004 and HG005 GIAB benchmark VCFs, and an HG002 precisionFDA submission with unprefixed contig names [41, 42]. Arm 2 contains 104 unrelated high-coverage 1000 Genomes participants, four from each of 26 populations [43]. Arms 1 and 2 use normalized, split records restricted to the ACMG gene spans (7.1 Mb of Ensembl 116 spans), with bcftools left-alignment and multiallelic splitting. For Arm 2, repeated panel-level INFO describing all 3,202 panel samples is removed from individual participant files, and the required population frequencies are represented once in a separate sites-only graph. Arm 3 processes the complete HG005 VCF without the region restriction (3,887,810 records after normalization); its result must equal the HG005 result from the restricted Arm 1 input. Arm 4 processes the complete

NB72462M VCF (5,063,417 records after normalization), described below. All arms use the ClinVar release from 2026-09-13, restricted to the same gene spans [44].

All consent and access conditions are simulated. In Arms 1–3, requesters represent clinical care (CC), disease-specific cardiovascular research (DS), and general research use (GRU). File-level permissions, participant withdrawals (four in the seeded cohort policies), and a restriction that limits records in 28 cancer-predisposition genes to clinical care determine the resulting release views. Arm 3 releases every record for CC and none for DS or GRU. Each participant is represented by a single-sample VCF, so no arm assesses selective masking of samples within a multi-sample VCF.

Four additional rule changes test where policy configuration or execution logic changes in each workflow: a withdrawal, a new requester purpose, replacement of the cancer-gene panel with a cardiac panel, and restriction of HFE C282Y. After each change, both workflows must release identical records on a fixture containing cancer, cardiac, other, and HFE records. Line counts exclude blanks, comments, and docstrings; they describe these implementations, not usability or maintainability.

Arm 4: fine-grained release from one complete VCF. In Arms 1–3, release views are determined mostly per file: a participant’s consent releases or withholds the whole file, and only the cancer-gene restriction selects within it, so Arm 3’s release views of the complete HG005 VCF were all or nothing. Arm 4 asks the same application question of the complete NB72462M VCF, a consumer WGS file from Nebula Genomics, with policies that produce different non-empty release views from the same file. The simulated consent permits clinical care and general research use, which includes disease-specific research, and the participant’s physician is a fourth requester. The policies combine purpose-specific gene-panel, genomic-region, and single-variant rules with the cancer-gene restriction, which still limits records in the 28 cancer-predisposition genes to clinical care through their gene links. The gene-panel rule limits disease-specific research to records whose REF span overlaps one of 40 cardiovascular genes. It prohibits the panel’s complement, declared as a SPARQL selection that fails closed when gene links are absent; a permission on the panel alone would also withhold the file and header resources from which the query starts. The genomic-region rule releases APOE (chr19:44,903,787–44,909,396, Ensembl 116) only for care by the participant’s physician, a purpose declared in the case vocabulary as narrower than clinical care. ODRL has no “everyone except” assignee, so restricting by purpose rather than by requester keeps the rule closed to any other clinical requester. Because a region is defined by coordinates on a named contig, the rule depends on the contig name written in the file; a file naming the contig 19 would escape a chr19 rule. The single-variant rule withholds one variant of uncertain significance, DSP chr6:7,569,314 C>G, from the GRU requester by assignee; it targets the variant’s SPDI identity and therefore holds however the file names chromosome 6. The conventional workflow applies the same rules in Python to every record of the VCF, so Arm 4 compares the number of records in each release view as well as the matches.

S5.3 Linking results, matches, and rule changes

Table S7 reports each linker separately. A seven-variant BRCA1 check agreed with NCBI SPDI outside repeats, while repeat-region spans differed between minimal trimming and NCBI’s contextual representation. The live tier confirmed 9,223 of 9,951 and 9,479 of 10,316 PGP rsID rewrites in 21 requests. All three unconfirmed identifiers inspected had merged rsIDs; the other unconfirmed cases were not resolved by that check. Ensembl’s live variation service returned HTTP 500 after three retries, and no partial linkset was published.

The RDF and conventional workflows returned identical requester-specific match sets, and every release view passed release validation (Table S8). P/LP matches numbered zero in Arms 1, 3, and 4 and eight in Arm 2. The complete HG005 VCF (Arm 3) returned the same 1,496

Table S7. Calls linked per linker, summed over each arm’s files. SPDI eligibility requires explicit alleles; 19 further HG002 calls in Arm 1 overlap genes but are ineligible for the SPDI linker. Arms 1–3 used the pre-release linkers; Arm 4 used v3.3.1, whose gene linker also links spanning-deletion (*) records, which the SPDI linker skips (15,032 in Arm 4).

Linker	Tier	Arm 1	Arm 2	Arm 3	Arm 4	ClinVar	Result
spdi	2	51,302	1,149,198	3,887,200	4,991,223	331,413	Every eligible call; joins the input VCFs to ClinVar across contig names; 87 s for the complete HG005 VCF
ensembl-genes-grch38	2	51,321	1,149,198	1,704,921	2,063,627	—	Reloads the 108 MB GFF3 per file: 19 of Arm 2’s 22 min
rsid-dbsnp	1	20,267	—	—	4,974,526	—	PGP VCFs’ rsIDs, rewritten unchecked
rsid-myvariant	3	18,702	—	—	—	—	92–93% of tier-1 links confirmed in 21 requests; the three unconfirmed rsIDs checked were merged
rsid-ensembl	3	0	—	—	—	—	Persistent HTTP 500; failed closed, no partial linkset

tuples as its gene-span slice in Arm 1. The additional cohort query joined a separate 68,635-site, 6.5M-triple frequency graph through SPDI; the conventional workflow’s position-and-allele lookup agreed for every requester. Storing the required frequencies once avoids approximately 857M triples of repeated panel INFO, which accounted for 93% of each participant’s graph before projection. This is reuse of shared information, not compression of an equivalent graph.

Table S8. Requester-specific participant–variant–gene matches; the RDF and conventional workflows agree in every cell. All: no policy; OP: care by the participant’s own physician (Arm 4 only); CC: clinical care; DS: cardiovascular disease-specific research; GRU: general research use. Triple counts exclude links; the rare-variant row also joins panel frequencies at AF < 0.01.

Arm	Files	Records	Triples	Matches by requester				
				All	OP	CC	DS	GRU
1: five gene-span slices	5	51,386	9.0M	7,211	—	2,878	2,988	1,987
2: cohort	104	1,150,288	67.8M	173,102	—	63,529	92,870	48,207
rare in the panel				4,154	—	1,621	2,198	1,144
3: complete HG005	1	3,887,810	662.4M	1,496	—	1,496	0	0
4: complete NB72462M	1	5,063,417	1,209.7M	1,382	1,382	1,382	722	983

In Arm 4, both workflows agreed on every match and on the number of records in each release view (Table S9), and all four release views passed release validation. The DS release view retained 722 of the 1,382 unrestricted matches and the GRU release view 983; none was P/LP. The release views withheld the six APOE records from every requester except the participant’s physician, the DSP variant from GRU, the cancer-gene records from both research purposes, and everything outside the cardiovascular panel from DS.

The record comparison is what exposed a linking defect. With VCF-RDFizer v3.3.0, Arm 4 on a 10,260-record slice of NB72462M (the gene spans, APOE, and a 100-kb control window) agreed on every match but not on released records: the GRU release view held 13 more records than the conventional workflow’s, and the DS release view 17 fewer. All 30 were records whose ALT is the spanning-deletion allele *. The gene linker skipped them, although * leaves the REF span unchanged, so they carried no gene links: the cancer-gene restriction never selected the 13 in cancer genes, and the gene-panel rule withheld the 17 in cardiac genes. Release validation could not detect this, because it evaluates the same links. Release v3.3.1 links * records by their

Table S9. Arm 4: what each rule withheld from the complete NB72462M VCF (5,063,417 records after normalization), per requester. The simulated consent permits clinical care and general research use. A record withheld by several rules is counted once, under the first; for DS, APOE and the DSP variant fall outside its panel. OP: the participant’s own physician; other abbreviations as in Table S8.

Rule	Selects records by	OP	CC	DS	GRU
Cancer genes (28): CC only	gene links	—	—	2,513	2,513
Cardiovascular panel (40): all DS sees	complement of the panel’s gene links	—	—	5,054,507	—
APOE: physician only	coordinates on chr19	—	6	—	6
DSP variant: not GRU	SPDI identity	—	—	—	1
Records released		5,063,417	5,063,411	6,397	5,060,897
Matches		1,382	1,382	722	983

REF span; symbolic alleles and breakends, whose extent is END or a mate, are still skipped and counted. The Arm 4 results above use v3.3.1, whose gene linker skipped no NB72462M record. In Arm 1, the same 13 records, and 11 from NG131FQA1I, had reached the research release views: 13 the GRU view and 24 the DS view. Rerunning Arm 1’s linking and policy evaluation with v3.3.1 left every match unchanged, removed those records from the release views (16,118 to 16,105 GRU records and 23,957 to 23,933 DS records), and its released-record counts matched the conventional workflow for every requester. Arms 2 and 3 were unaffected: the cohort files contain no * records, and HG005’s research release views were empty.

Restricting HFE C282Y required a policy edit without changing the policy engine of the RDF workflow; the conventional workflow required a small code and data edit (Table S10). The identity-targeted rule also selected HG002’s record on contig 6, which a literal **chr6** selector would miss. The other three rule changes were similarly sized data edits in both workflows, and all changed outputs agreed on the fixture.

Table S10. Authored content and rule changes in Arm 1, in non-blank, non-comment lines. After each change, both workflows returned the same requester-specific records. POA: DUO’s population-origins-or-ancestry purpose, outside the GRU permission used here.

	RDF workflow	Conventional workflow
<i>Initial authoring</i>		
Rules written	72: policy 41, query 31	131: shell 33, Python 98
Data written	28 gene IRIs; the query’s gene list (1 line)	Consent and requester tables (100 lines); gene list (81)
Linkers, selectors, and profile	Shipped with the tool	—
<i>Rule changes</i>		
A participant withdraws	Policy +2/ − 1	Data +2/ − 1
A requester with a new purpose (POA)	Purpose vocabulary +2	Data +4
The panel becomes 40 cardiac genes	Policy +40/ − 28 (gene IRIs)	Data +41/ − 29 (gene symbols)
One variant (HFE C282Y), clinical care only	Policy +6, no engine change	Code +3, data +9/ − 1

S5.4 Stage costs

Table S11 gives every stage time behind main Figure 6: conversion and linking once per arm, and for each requester the release view’s writing, validation, query index, and queries. Query times of 172–209 s across the arms are consistent with a dominant fixed ClinVar join, not generally scale-independent retrieval. Arm 2 initially required 20–27 min per release view because each line was checked against 109 rules. Combining the binding selections and filtering in parallel reduced this to 131–165 s; resolving each term’s ancestors once reduced it further to 59–80 s, with byte-identical views. Arm 3’s CC release view streamed in 38 min on two workers, one per input file, and peaked

at 13.1 GB resident memory and 34 GB disk. In Arm 4, on **vcf-bench-3**, converting NB72462M took 90 min (1.21B triples) and linking it 23 min; writing and validating each release view took 55–74 min, building each full view’s 1.26B-triple query index 83–84 min, and each query 180–185 s, or 146 s for the 1.5M-triple DS release view. Available memory never fell below 17.3 GB.

Table S11. Stage costs of the linked workflow for every arm and requester; the data behind main Figure 6. Conversion and linking are paid once per arm, and both include ClinVar. Write is the policy evaluation that streams the release view. Check is release validation, measured as the interval before the next requester’s evaluation, so it is not recorded for the last requester; for a non-empty view it includes building the view’s own index. Index is the build of the query index, and query the median of three runs over it. Records are those in the release view; triples are the query index’s, including ClinVar and links. Abbreviations as in Table S8.

Requester	Records	Index triples	Write <i>min</i>	Check <i>min</i>	Index <i>min</i>	Query <i>s</i>
<i>Arm 1: five gene-span slices: convert 3.3 min, link 1.4 min</i>						
All (no policy)	51,386	42,543,708	—	—	2.2	175
CC	19,625	37,514,190	0.53	0.47	2.2	174
DS	23,957	37,818,548	0.65	0.62	2.2	172
GRU	16,118	35,969,925	0.58	—	2.2	181
<i>Arm 2: cohort of 104: convert 31.1 min, link 22.1 min</i>						
All (no policy)	1,150,288	110,268,919	—	—	3.3	202
CC	421,991	65,787,093	0.98	2.9	2.5	181
DS	685,928	81,878,619	1.3	4.9	3.1	188
GRU	356,835	61,794,418	1.2	—	2.4	177
<i>Arm 3: complete HG005 VCF: convert 54.5 min, link 8.0 min</i>						
All (no policy)	3,887,810	701,489,520	—	—	45.3	209
CC	3,887,810	701,489,520	38.3	68.1	45.8	207
DS	0	33,428,173	28.0	1.0	2.2	0.06
GRU	0	33,428,173	28.0	—	2.2	0.06
<i>Arm 4: complete NB72462M VCF: convert 92.7 min, link 23.1 min</i>						
All (no policy)	5,063,417	1,255,334,983	—	—	83.2	184
OP	5,063,417	1,255,334,983	72.8	117.0	84.0	182
CC	5,063,411	1,255,333,543	74.3	116.6	83.8	181
DS	6,397	34,971,594	55.1	9.7	2.2	146
GRU	5,060,897	1,254,725,280	73.4	—	83.5	181

S6 Reproducibility and execution

S6.1 The base campaign

The *base campaign* is the set of benchmark runs that produced the main text’s fidelity and scalability results: 143 runs (*cells*) in 12 experiment families, all executed with the published VCF-RDFizer v3.1.0 release on two identical hosts, **vcf-bench-1** and **vcf-bench-2** (Sections S6.2 and S6.3). Each family is one script of the benchmark harness in the testing repository, numbered as below; scripts 00 and 02 only record the environment and derive the ladder inputs, and are not counted. Every family ran as a whole on one host, never split across hosts, with no other benchmark running, and every cell wrote its command, commit, image, exit status, and timings to a fresh output directory. The families, with their inputs and what each varies, are:

Records scaling (04). Prefixes of HG005 with 10,000, 100,000, and one million records, three replicates each, and the complete VCF (3,856,856 records) once (10 cells); main Section 4.3.1 and Figure S3.

- Sample representation (03).** 10,000 records of the 1000 Genomes chromosome-20 call set at 1, 4, 16, 64, 256, 1,024, and 2,504 samples under both sample profiles (14 cells), three timing replicates at 1 and 2,504 samples in each profile (12), and four anchor cells on a single-sample and a multi-sample fixture; main Section 4.3.1.
- Storage mode (01).** Plain against space-optimized storage on 100,000 HG005 records, three replicates each, and on a 269M-triple test file once each (8 cells); main Section 4.3.2 and Figure S3f.
- Corpus breadth (05).** HDT and COTTAS construction on the first 250,000 records of eight public inputs and on the complete HG005 VCF (10 cells); main Section 4.3.2 and Figure S4.
- Representation axes (07).** Raw against structured INFO on 100,000 HG005 records and on HGSVC2, three replicates each; basic against structured header output; and fixtures declaring VCF 4.1 to 4.5 or no version (20 cells); Sections S7 and S10.
- Difficult inputs (09).** Eleven awkward fixtures, one conversion with the three demonstration linkers, and a custom-mapping cell (16 cells); Section S7 and Table S15.
- Covering set (11).** Ten configuration rows on a 1,000-line fixture that together cover every value and every pair of values of seven options, each validated with shapes (10 cells); Section S10.
- Modes (12).** Each operating mode run on its own on the 1,000-line fixture and its artifacts: TSV, conversion, validation, compression, decompression, and HDT and COTTAS indexing (7 cells); Section S10.
- Feasibility (10).** One million HG005 records under three configurations, each with requested memory ceilings of 8, 16, and 31 GB (9 cells); Section S10.
- Robustness (08).** Two identical conversions of 100,000 HG005 records, a convert-compress-decompress round trip, an index build and rebuild, and the 113-fault mutation score with the default and with all shape profiles (9 cells); main Section 4.1 and Section S4.
- Equivalence (06).** The 10,000-line (9,986-record) fixture in both storage modes and both sample profiles, validated on four SPARQL engines over three RDF artifacts, two HDT strategies, and two required refusals (8 cells); Section S10.
- Query cost (13).** The thirteen source-comparison queries against cyvcf2 on the 10,000-line fixture (0.96M triples) and on 100,000 HG005 records (17.1M triples), three replicates each (6 cells); main Section 4.3.3 and Section S9.4.

The 78 validation runs of the base campaign come from five of these families: Difficult inputs (20), Covering set (23), Equivalence (16), Query cost (18), and Modes (1).

The base campaign took 96.16 h of measured wall time (Table S12). Every asserted expectation held, but not every cell completed successfully. The truncated-gzip and sites-only fixtures exited non-zero and are recorded as outcomes; the two HDT single-strategy cells with COTTAS or with space-optimized storage were required refusals; and three cells were skipped by a guard: a 2,504-sample input under the expanded profile in two families (the 1000 Genomes call set in Corpus breadth and the multi-sample anchor in Sample representation), because its sample columns exceed the volume’s sample limit, and the custom-mapping cell, because no custom rules file was supplied. The two mutation-score cells record the host-side test-harness commit rather than a conversion-container execution.

Runs outside the base campaign are reported separately and are not counted in its totals (Table S13): the linked workflow, regional retrieval with tabix, large-graph retrieval on `vcf-bench-3`, and two additional validation runs (a default-profile mutation rerun and the consumer WGS validation run on 250,000 NG131FQA1I records, repeated with the corrected validator), and an N-Triples-only rerun of the 100,000-record query-cost workload. An earlier pre-release campaign and the host-calibration runs are archived with the results but not reported.

Table S12. Base campaign by experiment family. Outcomes count cells: OK; Rec., recorded (a non-zero exit without an asserted outcome); Ref., refused (an expected rejection); Skip., skipped (a guard prevented execution). Hosts b1 and b2 are `vcf-bench-1` and `vcf-bench-2`; sections without an S prefix are in the main text.

Experiment	Host	Cells	Hours	Outcome				Question answered
				OK	Rec.	Ref.	Skip.	
<i>Conversion, scaling, and storage</i>								
Records scaling	b1	10	10.99	10				Cost growth with records (§4.3.1)
Sample representation	b2	30	19.69	29			1	Profile growth with samples (§4.3.1)
Storage mode	b1	8	12.87	8				Peak workspace, plain vs space-optimized (§4.3.2)
Corpus breadth	b2	10	26.90	9			1	HDT and COTTAS on ten public inputs (§4.3.2)
<i>Input and configuration coverage</i>								
Representation axes	b1	20	4.82	20				VCF 4.1–4.5; INFO and header costs (§S7, §S10)
Difficult inputs	b1	16	0.10	13	2		1	Eleven awkward VCFs; demonstration linkers (§S7)
Covering set	b1	10	0.35	10				Every option value and pair (§S10)
Modes	b1	7	0.03	7				Each operating mode alone (§S10)
Feasibility	b1	9	9.03	9				Memory ceiling × configuration (§S10)
<i>Validation and retrieval</i>								
Robustness	b1	9	0.25	9				Determinism, round trip, mutation score (§4.1, §S4)
Equivalence	b2	8	6.84	6		2		Four engines, both profiles, three artifacts (§S10)
Query cost	b1	6	4.27	6				SPARQL vs cyvcf2 on identical work (§4.3.3)
Total		143	96.16	136	2	2	3	

S6.2 Builds and dependencies

The base campaign used VCF-RDFizer v3.1.0, commit d3b34d5. The Docker image is `ecrum19/vcf-rdfizer:v3.1.0`, with digest

`sha256:1904e96dde12ab2e2e70d8ee1267765c293ab100a8bd1b14d5b009b2bf8e34aa`

Image rebuilding was disabled during execution. The image bundles the VCF Core SHACL shapes at vocabulary commit 5bfd19d; this snapshot is distinct from the separately versioned vocabulary release [16].

The pinned dependencies are RMLStreamer 2.5.0, hdtc 1.1.0, pycottas 1.1.0, DuckDB 1.5.5, cyvcf2 0.34.0, pyshacl 0.30.1, Comunica 5.3.0 for N-Triples, @comunica/query-sparql-hdt 5.0.1, the COTTAS Comunica engine 0.1.0, bcftools from the base distribution, and QLever pinned by image digest. The image also fixes the hdt-cpp revision used to compile `rdf2hdt` and `hdt2rdf`. Later experiment families are identified separately in Table S13. Use-case Arms 1 and 2 ran uncommitted `feature/spdi-linker` trees on v3.2.0, identified per cell by a diff or tree hash; Arm 3 used 578c2be. The subsequent v3.3.0 release adds refusals for an empty selector list and an RDF input without VCF triples, neither of which occurs in these use-case inputs.

[AUTHOR QUERY: Archive the uncommitted arms 1–2 source trees and record the complete v3.3.0 image digest.]

The 657.4M-triple conversion/retrieval graph and the 662.4M-triple normalized use-case graph are different artifacts, not alternative counts of the same graph.

Table S13. Builds and assessment boundaries of each experiment family. A reproducible state requires the recorded code, local changes, dependencies, and external resources.

Family	Recorded state	Assessment boundary
143-cell base campaign	v3.1.0, d3b34d5; full image digest above	Conversion, artifacts, fixtures, 984 applicable paired comparisons, and configuration tests
Regional retrieval	3da1a5a	Later runner with tabix; 11,160 regional executions
Large-graph retrieval	v3.1.0 graphs; runners 245fe1f, 5ed75c8	Separate host; thirteen summary queries without SHACL
Linked use case	v3.2.0 conversion; uncommitted pre-release trees on f266db5 in arms 1–2; 578c2be in arm 3	SPDI, gene links, and policy views; v3.3.0 (7ab2300) is the subsequent release, not the exact state of every run
Use-case Arm 4; Arm 1 rerun	Linking and policy evaluation from 29a81f6 (v3.3.1); images 3.3.0 (Arm 4) and 3.2.0 (Arm 1)	Record-level comparison; Arm 4 on vcf-bench-3, Arm 1 on vcf-bench-1; derive and convert reused from earlier runs
Additional validation runs	v3.1.0 (d3b34d5, image sha256:1904e96d...34aa) on vcf-bench-2	Default-profile mutation rerun; consumer WGS validation run (250,000 NG131FQAII records) on QLever, expanded profile, rerun without shapes under v3.1.0’s size gates
Consumer WGS validation rerun	Tree committed as 3994e94, validation code as released in v3.3.1; scratch image sha256:29c371ad...5317 on vcf-bench-2	Same slice and graph; QLever, expanded profile; default shapes in 117 batches on four workers
N-Triples-only rerun	v3.1.0 (d3b34d5, image sha256:1904e96d...34aa) on vcf-bench-2	100,000 HG005 records; conversion to gzip-framed N-Triples only, then the query-cost validation on QLever without SHACL; three replicates each
Shared-input converter comparison	Harness e3743590; VCF-RDFizer v3.3.1 image sha256:3ad71b1a...2993; the other converters as in Table S2, on vcf-bench-1	100,000 HG005 records and the 16-sample, 10,000-record ladder rung; Q1–Q8, three replicates (Section S2)

S6.3 Hosts, measurements, and execution status

The base experiments used two separate hosts, **vcf-bench-1** and **vcf-bench-2**, each with an Intel Xeon Silver 4309Y at 2.80 GHz (8 cores), 33.6 GB RAM without swap, a 210 GB SSD, Ubuntu 24.04.4 LTS, kernel 6.8.0-85-generic, Docker 29.7.2, and Python 3.12.3. Each complete experiment ran on one host without concurrent benchmark jobs. Calibration produced identical triple sets, byte-identical COTTAS artifacts and HDT indexes, and differences only in N-Triples/HDT ordering and the HDT `dcterms:issued` timestamp. Conversion nevertheless took 26 versus 37 s, and validation 156 versus 208 s, so timings are compared within hosts. Large-graph retrieval ran separately on **vcf-bench-3**, with the same CPU, 8 cores, 31.3 GiB RAM without swap, Ubuntu 22.04.5, and kernel 5.15. It used the v3.1.0 graphs and runners 245fe1f for N-Triples and 5ed75c8 for HDT and COTTAS.

Each cell records its command, host, commit, image, exit status, and wall time in a fresh output directory. Stage reports record resident memory, triple counts, artifact sizes, validation outcomes, and transient workspace separately. Performance denominators use uncompressed VCF bytes; Table S14 instead reports compressed download sizes. Replicated comparisons describe variability, whereas single-run corpus results do not estimate run-to-run variation.

Commands, normalized answers, and comparison reports are maintained in the testing repository; the complete evaluated snapshots still require a persistent archive. Tagged releases are distributed through PyPI, Conda-Forge, and Docker Hub under the MIT License, with instal-

lation instructions, examples, release notes, and issue tracking [45]. The unit-test suite covers Linux, macOS, and Windows; the reported benchmarks used Ubuntu.

S7 Input and version coverage

Table S14 identifies the ten public inputs; download commands and canonical filenames are maintained in the testing repository.

Table S14. Public corpus and evaluation roles. Names are canonical `.vcf.gz` stems; assemblies and sample counts were checked against the headers. Sizes are approximate compressed downloads. WGS: whole-genome sequencing; each file is a VCF of variant calls, not a genome sequence.

File	Provider	Assembly	Samples	MB (gz)	Evaluation role
NG1N86S6FC	Sequencing.com	GRCh38	1	379	Consumer WGS
NG131FQA1I	Dante Labs	GRCh38	1	224	Consumer WGS
NB72462M	Nebula Genomics	GRCh38	1	341	Consumer WGS
60820188475559	PGP upload	GRCh37	1	325	SNP-only profile
60820188474283	Dante Labs	GRCh37	1	222	Consumer WGS
OG00R_HG002	precisionFDA	GRCh38	1	69	GIAB Truth Challenge v2
1000G_phase3_chr20	1000 Genomes	GRCh37	2,504	327	Phased cohort
HGSVC2	HGSVC2	GRCh38	32	31.5	Multi-sample SVs
HG004_GRCh38	GIAB v4.2.1	GRCh38	1	149	Benchmark VCF, chr1–22
HG005_GRCh38	GIAB v4.2.1	GRCh38	1	139	Benchmark VCF, chr1–22

Versions and corpus processing. Fixtures declaring VCFv4.1–4.5, plus one with no `##fileformat`, all converted. Their generated RML mappings differ only in the file class, from `vcfc:VCF41File` to `vcfc:VCF45File`. Version-specific fields are handled through the data: the two-record VCFv4.5 fixture carries LAA and LAD, and the VCFv4.4 fixture carries PS, yielding 252.5 and 210 triples per record, respectively. This demonstrates retention of those fields, not interpretation of every version-specific construct; local-to-global allele resolution is not implemented. The undeclared-version input uses the documented fallback.

Nine corpus inputs were processed with headers retained: the first 250,000 records of eight files, and HG005 whole. Their HDT and COTTAS outputs passed decoding/count checks. The harness skipped the complete 2,504-sample 1000 Genomes input because its expanded-profile corpus guard excludes files with more than 1,000 sample columns. Both sample profiles were instead tested on 10,000-record slices containing all 2,504 samples. Separately, the tool refuses an expanded conversion when its estimated workspace exceeds three quarters of free space and suggests condensed output. Neither guard is evidence that the complete cohort was converted.

Difficult inputs. Nine of eleven fixtures converted and completed paired validation (Table S15). The truncated gzip was refused during decompression, while a malformed sites-only fixture produced RDF but did not complete validation. The separate sites-only frequency graph in the linking experiment demonstrates a working frequency join, not completion of the missing paired-validation test.

Table S15. Difficult-input fixtures. Triple counts describe the produced graph; an intended refusal is not a fidelity result.

Fixture	Triples	Validation	Note
CRLF line endings	262	PASS	No stray <code>\r</code> in the last field’s literal
Symbolic alleles	453	PASS	<code></code> not treated as a base string
Breakend ALT	417	PASS	Breakend ALT syntax preserved
Mixed ploidy	350	PASS	Ploidy 4 then 3 followed the data
Half and haploid calls	446	PASS	Four genotype cases stayed distinct
Missing values	321	PASS	Missingness follows the field-specific representation
FILTER variants	486	PASS	<code>PASS</code> , <code>.</code> , and a failed code are three states
Declared FORMAT unused	293	PASS	Header declaration represented although never used
Zero data records	189	PASS	Header-only input
Sites only	245	incomplete	Graph and HDT artifact produced; validation did not complete
Truncated gzip	—	refused (intended)	Refused at decompression in 0.58 s with a diagnostic

S8 Defects, corrections, and remaining gaps

This section retains diagnostic evidence that is not visible in the aggregate pass counts. Corrected outputs are used where stated; unsuccessful runs and unresolved failures have not been relabeled as passes.

Declared samples in a header-only input. The emitter learned the file identity from the first record and omitted `vcfc:SampleSet` and `vcfc:VCFSample` when there were no records. Q9 and Q10 detected the omission. Deriving the sample set from the header increased the graph from 181 to 189 triples and allowed validation to complete. The preflight oracle also incorrectly expected a `vcfc:sampleId`, which belongs to a per-record `vcfc:SampleCall`; it was corrected to expect no calls in an empty file.

Allele references without allele resources. Three covering-set rows combined raw INFO with expanded samples, whose genotypes referenced alleles emitted only by the structured-INFO path. A 1,000-record test contained 1,972 dangling references and produced 5,916 shape violations, although query counts did not follow the references. The emitter and census expectations were corrected to include alleles when either representation requires them. All ten rows then passed; the three affected rows gained 10,846 triples each, and the other seven remained byte-identical. Regression tests cover all four INFO/sample combinations and require every referenced allele to be described.

Metadata retained after a file was withheld. The policy plug-in initially removed the correct records but retained a withdrawn file’s header in every view. Its own check detected the problem. The ownership rule now determines the dependent context removed with each withheld resource rather than treating record subtrees as the complete release boundary.

Memory guards and query heaps. The v3.1.0 shape gate used packaged bytes although `pyshac1`’s in-memory cost depends on the graph. The same 170,935,101-triple graph was admitted as 390.7 MB COTTAS and killed at 30.7 GiB resident memory on a 31.3 GiB host, but skipped as 756.6 MB gzip-framed N-Triples or 1.18 GB HDT. In full mode, the gate instead used VCF bytes, admitting the 58.2M-triple NG131FQA1I graph from a 63.8 MB input and exhausting memory; the diagnostic rerun excluded shapes. The corrected gate counts triples rather than packaging-dependent bytes. The Comunica-backed endpoints also exhausted the JavaScript heap

while approximately 25 GB host memory remained free; the heap ceiling is now configurable. Both corrections are released in v3.3.0, do not change conversion output, and leave the failing runs in the evidence record. Release v3.3.1 lowers the whole-graph limit from 50M to 10M triples, about 24 GB at the measured 2.4 GB per million triples, and validates record-local shapes in batches whose memory follows the batch rather than the graph; with it, the NG131FQA1I graph passed the default shapes (Section S4.2).

COTTAS execution and timeout handling. The initial pycottas/RDFLib path completed preflight and Q1–Q4 on condensed output but stalled on Q5. Two attempts ran for approximately 41 and 10 h at about 190% CPU before being stopped; a 1,800-second query limit and 14,400-second budget did not interrupt the second in-process query. Both runs are archived. The replacement DuckDB-backed Comunica path reduced Q5 to 2–5 s and the thirteen-query workload on 0.96M triples to 38 s, against 1,402 s through RDFLib. Completed comparisons use the replacement path, with query execution placed where a timeout can terminate it.

Shape severity and missing tokens. The SHACL report parser initially treated recommendations as blocking and expected a violation format that pyshacl did not emit. It was corrected to classify severity and block only `sh:Violation`, rather than reject valid INFO declarations without `Source`. The missing-token preflight also flagged legitimate dots: `Number=.` denotes variable cardinality, and `.` can encode a missing genotype. Excluding `vcfc:fieldNumber` and `vcfc:genotypeString` reduced 20 rows to ten header-attribute values carrying the same Number declaration. A further correction excludes that attribute while retaining checks on others, but postdates v3.1.0. Its reports in the base campaign therefore retain 608 checks, 599 passes, and nine flags with ten rows each; those flags do not represent malformed RDF.

Incomplete sites-only coverage. The fixture declared `FORMAT` without sample columns, which bcftools rejects. It produced 245 triples and an HDT artifact, but both paired validations ended with `EXECUTION_FAILED`. This is incomplete fixture/oracle coverage, not a failed semantic comparison or a passed sites-only validation. A conformant fixture remains necessary.

Legacy measurement corrections. An earlier six-file benchmark under-reported partitioned compression times; commit `fb48e50` corrected timing, and affected historical durations were reconstructed from logs and timestamps. Its input-size column labeled `.vcf.gz` also held uncompressed bytes: NG131FQA1I contained 1,215,509,730 bytes, matching 1215.5 MB. The legacy HDT reduction of 92–95% used uncompressed N-Triples as its denominator, unlike the present 1.36–1.82 ratio against gzip-framed N-Triples. Those legacy tables are excluded from the current analysis rather than presented as directly comparable compression results.

Oracle failures on a consumer WGS VCF. The v3.1.0 oracle did not count phase sets and compared `QUAL` as written, so the consumer WGS validation run first reported three mismatches on a graph whose record fields equal the VCF text (Section S4.2). Release v3.3.1 corrects both and adds census expectations, checked against the emitters by unit tests, for structural-variant events, confidence intervals, and gVCF reference blocks; rerun with it, the same graph passed all thirteen comparisons. Local-allele constructs remain outside the oracle. These are distinct from the fixed sample, allele-resource, and ownership defects above.

S9 Conversion, storage, and retrieval measurements

S9.1 Record and sample scaling

On the HG005 record ladder, triple production had a fitted exponent of $+0.999$ (95% CI $[+0.991, +1.007]$), at 170.9–171.9 triples per record (Figure S3a). Artifact bytes, peak workspace, and end-to-end time had exponents of $+1.033$, $+1.025$, and $+1.063$, respectively. At 10,000, 100,000, and one million records, end-to-end times were 40.7, 423.7, and 5,476.0 s, with replicate standard deviation at most 13.5 s. Mapping-stage resident memory remained 1.0–1.7 GB across this hundredfold input increase. RMLStreamer itself took 12.2–41.4 s; HDT construction dominated total time, rising from 38% of it at 10,000 records to 82% for the complete VCF (Figure S3e). These observations describe mapping memory on this ladder, not bounded memory for every downstream component.

The sample ladder holds 10,000 records and INFO fixed while varying S from 1 to 2,504 (main Figure 7). Its triple counts are exactly $1,359,498 + 250,005S$ expanded and $1,439,498 + 5S$ condensed, reflecting 25 repeated triples per sample per record and five declaration triples per sample in the expanded path. The expanded fitted exponent of $+0.795$ reflects the fixed record layer at low sample counts. At 2,504 samples, expanded output contains 627,372,018 triples, $432\times$ condensed output. The byte ratios instead are $322\times$ for gzip-framed N-Triples and $69\times$ for HDT. Condensed HDT grows from 10.2 to 59.2 MB ($5.8\times$), despite only 0.9% triple growth, because its vectors lengthen. It becomes smaller than uncompressed VCF from 256 samples; at one sample the sizes are 10.2 versus 1.5 MB, and at 2,504 they are 59.2 versus 101.6 MB. Expanded HDT reaches 4.10 GB, approximately $40\times$ the VCF, with an end-to-end time of 15,269 s against 250 s condensed. Extrapolating expanded workspace to the full 1,812,841-record call set gives roughly 2 TB against the 189 GB volume available to that estimate, so the run is refused; this is an estimate, not a completed full-cohort measurement.

S9.2 Storage mode and optional artifacts

Plain and space-optimized storage produced identical triples and final artifact sizes within one 4 kB filesystem block. Space-optimized storage reduced peak workspace from 2.98 to 0.325 GB ($9.19\times$) on 100,000 records and from 43.40 to 5.65 GB ($7.68\times$) on the 1,155,741-record NG131FQA1I prefix **test-larger**. The end-to-end time costs were 5.2% and 3.2%, respectively (Figure S3f). On the replicated input, mapping time met a prespecified $\pm 10\%$ equivalence margin: observed difference $+0.0\%$, with a 90% bootstrap CI of $[-1.9\%, +0.4\%]$.

Across the corpus, COTTAS occupied 0.37–0.55 times gzip-framed N-Triples and 0.27–0.37 times HDT, while HDT occupied 1.36–1.82 times gzip-framed N-Triples (Figure S4). COTTAS took 1.5–3.0 times longer than HDT to build, and peak workspace ranged from 4.7 to 30.3 times the uncompressed VCF input. The v3.1.0 run on the complete HG005 VCF produced 657.4M triples in 16.07 h, including 20,911 s for HDT and 31,230 s for COTTAS, approximately 14.5 h together. Its 146.8-second RML stage excludes emitters and other conversion work. The retained HDT (4.63 GB), HDT index (4.74 GB), and COTTAS (1.50 GB) formed a measured 10,879,410,176-byte output tree, or 10.88 GB; rounded components sum to 10.87 GB. This is $78\times$ the 139 MB compressed download, but includes alternative representations and an index rather than a minimum RDF footprint. Gzip-framed N-Triples alone occupied 2.89 GB.

The later v3.2.0 use case produced 662.4M genomic triples in 53 min, or 54 min including ClinVar, before linking, release construction, checking, and indexing. Its inputs, version, and selected work differ from the 16.07-hour run; subtracting the times would not estimate a speed-up. An archive requires its selected RDF artifact, whereas a query deployment also requires its backend’s index and loading workspace.

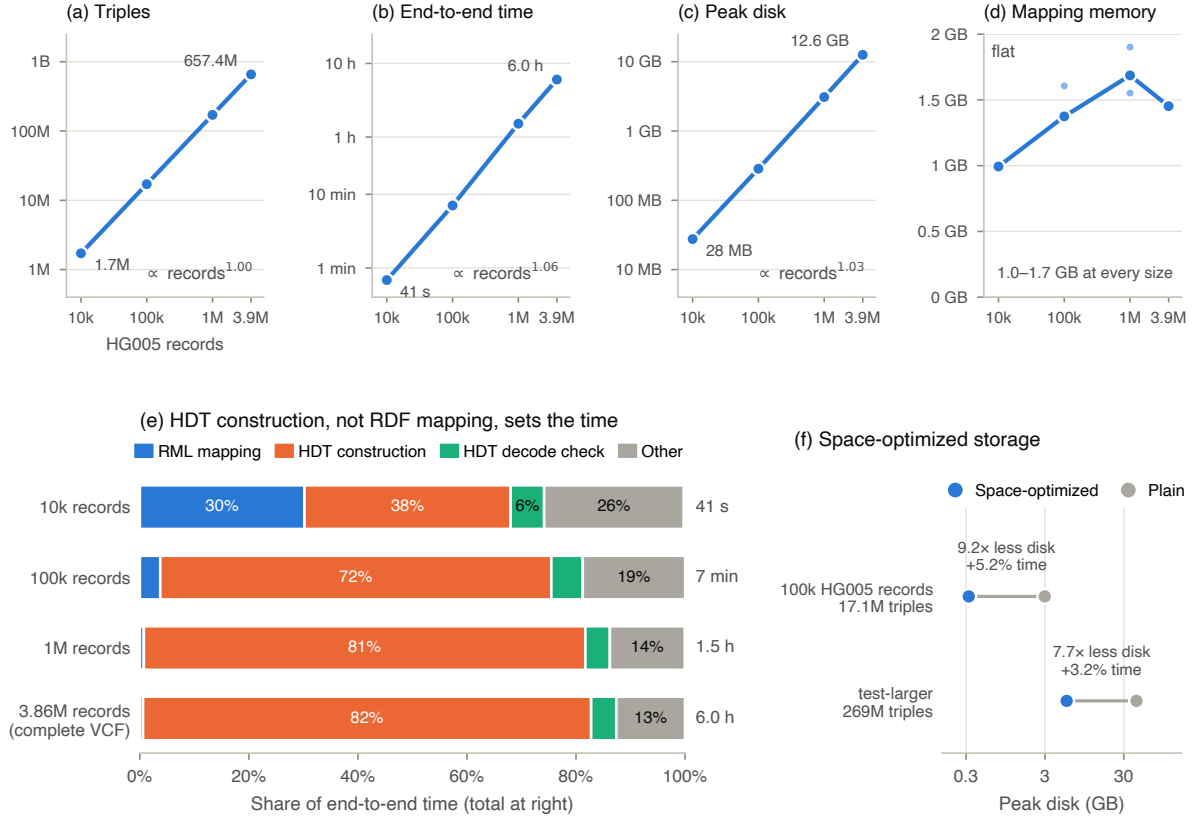


Figure S3. The record ladder: conversion scales linearly with records, while mapping memory does not grow. (a)–(d) Triples, end-to-end time, peak disk, and peak RMLStreamer memory for 10,000, 100,000, and one million HG005 records (medians of three replicates; light dots are the replicates) and the complete VCF of 3,856,856 records (one run), on log scales except (d); each exponent is a least-squares fit over all runs. (e) Share of end-to-end time by stage, with the total at right: RML mapping is the RMLStreamer stage; HDT construction covers chunk streaming, chunk builds, merging, and indexing; other is the remainder, including TSV conversion and the direct emitters. (f) Peak disk of plain and space-optimized storage on two inputs; triple counts and final artifacts agree between the modes. All runs use v3.1.0 on `vcf-bench-1`.

S9.3 Query workloads and physical access paths

On 17.1M triples, QLever’s thirteen-query totals from N-Triples, HDT, and COTTAS were 17.0–17.1s, differing by less than 1%; individual queries differed by at most 0.07s (main Figure 8c). These are queries over QLever’s index, not native artifact access. On the separate 0.96M-triple fixture, mean totals were 1.1s for QLever, 23.4s for Comunica over N-Triples, 38.2s for the COTTAS engine, and 45.1s for the HDT-backed engine, with answer agreement in every run.

For individual questions on 17.1M triples, full `cyvcf2` parses took 11.5s, or 12.3s for the three sample-level questions, against 0.005–4.62s after QLever indexing. The resulting 2.5–2300× query-time ratio excludes the 22.8-second index build and conversion. The tested individual cold-start questions therefore favored the parser. One parsing pass answering all thirteen questions took 12.4s, against 17.1s of indexed queries before setup.

The large-graph workload took 1.13, 1.00, 0.96, and 0.97s per million triples at 0.96M, 17.1M, 171M, and 657M triples, respectively (Table S16). Index construction took 1.27–1.35s per million above the smallest graph. On the complete HG005 VCF, the queries took 634.6s after an 868.0-second build, with replicates within 0.7% and every answer equal to the parser’s. Query times from the three artifacts differed by only 0.3%, while one-time decoding costs differed substantially. Both native compressed Comunica paths stopped at Q6 on 171M triples despite a 24 GB heap;

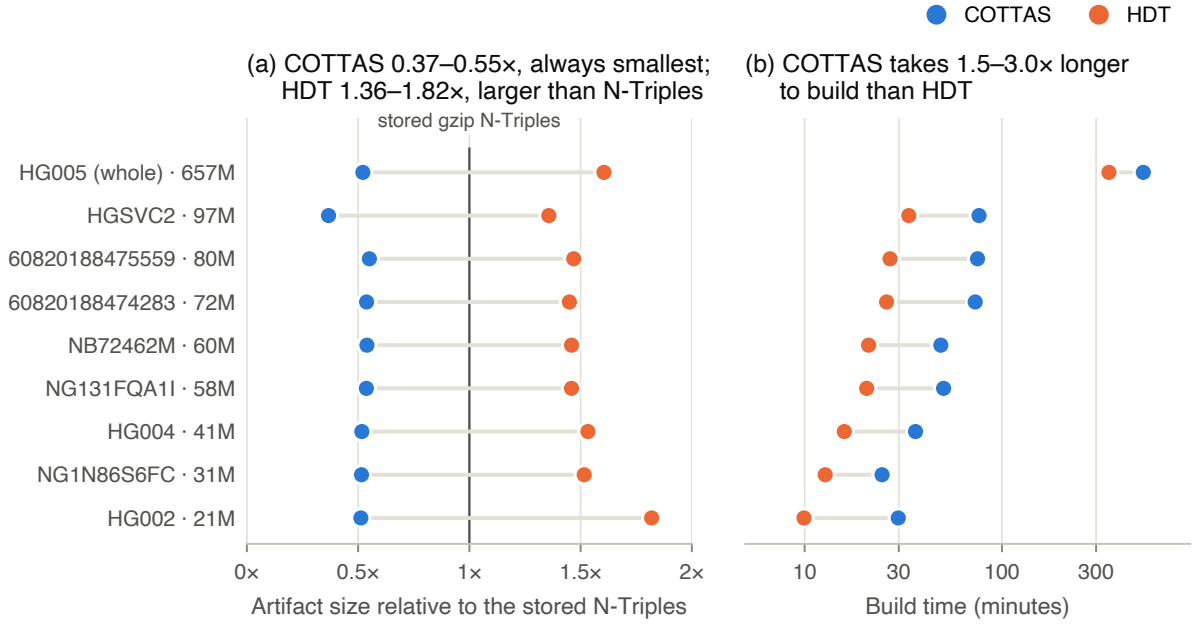


Figure S4. Corpus artifact sizes and construction times. Eight completed inputs use 250,000 records, and HG005 is processed whole; the expanded whole-cohort cell was skipped. (a) Artifact bytes relative to gzip-framed N-Triples. (b) Construction times on a logarithmic scale. Each input has one run, and all produced artifacts passed decoding/count checks; full filenames are in Table S14.

their reported totals cover only five answers. On those five, COTTAS was $16.7\times$ faster than HDT, compared with less than $1.2\times$ on the small fixture, so small-graph rankings do not establish large-graph behavior.

Table S16. Retrieval at scale on v3.1.0 HG005 graphs. Smaller graphs ran on `vcf-bench-1` and larger ones on `vcf-bench-3`, so normalized times keep host differences. Completed answers were compared with `cyvcf2`; setup is reported separately.

Graph	Triples	Engine	Time (s)	Outcome
<i>Does SPARQL scale to a complete VCF? (all thirteen questions)</i>				
Fixture	958,919	QLever	1.1	1.13 s per million triples
100 k records	17,098,746	QLever	17.0	1.00 s per million
1 M records	170,935,101	QLever	163.3	0.96 s per million
3.86 M records, complete	657,425,805	QLever	634.6	0.97 s per million; setup 868.0 s; replicates within 0.7%
<i>Does the artifact matter? (complete VCF, one engine, three packagings)</i>				
from N-Triples	gzip-framed		634.6	Decode 179–192 s; setup 868.0 s
from HDT	—		636.3	Decode 955–972 s; setup 861.8 s
from COTTAS	—		636.6	Decode 592–601 s; setup 868.4 s
<i>Do the other engines get there at all? (1 M records, 170,935,101 triples)</i>				
COTTAS engine	5 of 13 answered		743.1	Native allocation failure on Q6
HDT engine	5 of 13 answered		12,432.2	Q5 answered at the one-hour per-query ceiling (3,600.1 s); Q6 failed

Indexed regional access. The 11,160 regional executions produced no failures or answer disagreements (main Figure 9; Table S17). On 17.1M triples, QLever took 9.6–11.1 ms across 1 kb–10 Mb windows, while tabix-indexed region queries with `cyvcf2` and `bcftools` took $35.7\times$ and $7.5\times$ longer on 10 Mb windows than on 1 kb windows. At 1 kb, those region queries were $2.9\times$ and $2.2\times$ faster; QLever overtook `cyvcf2` by 1 Mb and led both by $10.6\times$ and $2.9\times$ at 10 Mb. QLever

setup took 23.2s, against 0.48s for bgzip and tabix. Figure S5a separates the five questions: QLever’s time depended on the question (2.6–16.2 ms) but hardly on the window size, and for the record count it was faster than tabix-indexed region queries with either tool at every window size. In-memory engines required 896–1005 ms per 1 kb window on the smaller graph, against QLever’s 18.8 ms, and were not carried further (Figure S5b). These comparisons use the same POS-based record selection rather than including interval-overlap records outside the requested window.

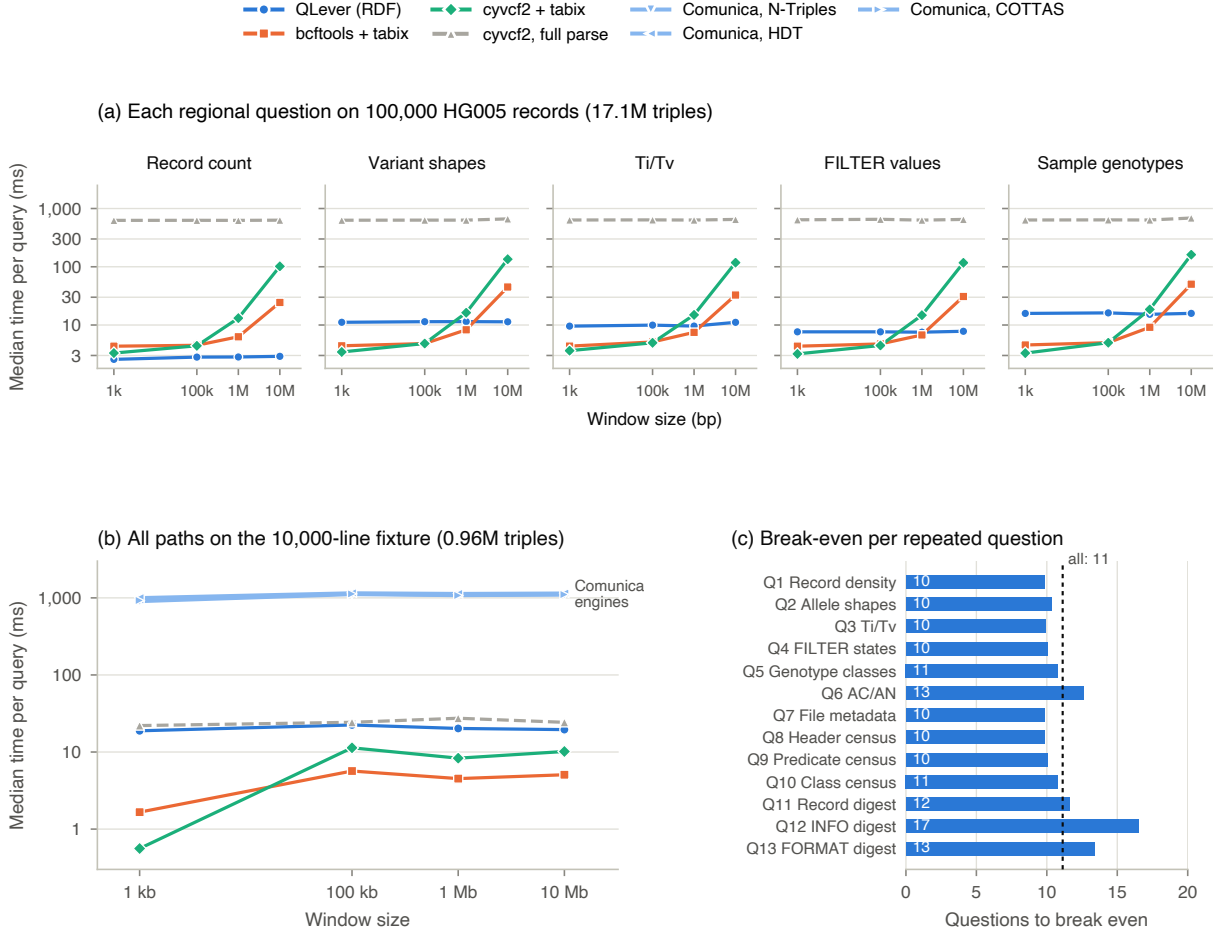


Figure S5. Regional and repeated-question retrieval in detail; the data behind main Figure 9. (a) Each of the five regional questions on 100,000 HG005 records (17.1M triples), median time per query by window size; main Figure 9c shows their median. (b) All seven execution paths on the 10,000-line fixture (0.96M triples), including Comunica over N-Triples, HDT, and COTTAS, which were not carried to the larger graph. (c) The break-even when a single whole-file question is repeated, from that question’s own cyvcf2 parsing and QLever query times in the N-Triples-only rerun; the dashed line marks the 11 questions of main Figure 9a. Setup is excluded in (a) and (b) and included in (c); it comprises conversion to the gzip-framed N-Triples that QLever indexes, and QLever indexing.

S9.4 Setup amortization

For an equivalent repeated workload with a positive per-query saving,

$$N_{\text{break-even}} = \frac{\text{additional RDF setup}}{\text{VCF time per question} - \text{indexed RDF time per question}}. \quad (2)$$

A three-replicate rerun on `vcf-bench-2` measured this scenario directly with the base campaign’s release, image, and 100,000-record input. It converted the records only to the gzip-framed N-Triples that QLever indexes (95.1–96.6s, mean 95.7s), then repeated the query-cost validation

Table S17. Regional retrieval on the 17.1M-triple graph of 100,000 HG005 records, excluding setup; the data behind main Figure 9c. Entries are medians in milliseconds over five questions’ window medians and three repetitions. Tabix-indexed region queries use twenty windows per size; full parses of the unindexed VCF use three. All paths use the same POS-based selection; the bottom row gives the median records per window.

Execution path	1 kb	100 kb	1 Mb	10 Mb
QLever (RDF)	9.6	10.0	9.6	11.1
bcftools + tabix	4.3	4.8	7.5	32.7
cyvcf2 + tabix	3.3	4.8	14.9	117.8
cyvcf2, full parse	635.2	638.0	632.0	649.9
<i>Records per window</i>	<i>2</i>	<i>143</i>	<i>1,262</i>	<i>13,143</i>

on them: QLever indexing took 25.0s, a full cyvcf2 parse a median 12.28s per question, and a QLever query a mean 1.43s, with all thirteen answers equal in every replicate. These give $(95.7 + 25.0)/(12.28 - 1.43) \simeq 11$ questions. The base campaign’s conversions also built HDT, which querying through QLever does not need; with it, the setup was 446.5s and the break-even 44 questions. The estimate also assumes a full parse of the VCF for every question at the measured question mix, not a single parsing pass computing all answers. No universal break-even count follows, and update costs for changing external evidence or policies were not measured. Main Figure 9a shows the cumulative costs behind this estimate; repeating a single question breaks even after 10 (seven of the thirteen questions) to 17 (Q12, the INFO digest) repetitions (Figure S5c). For storage, the gzip-framed N-Triples were 75.8 MB against a 3.1 MB gzipped VCF for these records, and 2.89 GB against 139.3 MB for the complete HG005 VCF (main Figure 9b); the gzipped VCF sizes were measured on the benchmark hosts because the run records hold uncompressed input sizes.

S10 Configuration coverage, operating modes, and memory feasibility

A ten-row covering set exercised every value of seven axes: sample representation, INFO representation, storage mode, RDF compression, output representations, artifact compression, and HDT strategy. Every row completed validation, including shapes. Pairwise coverage excludes only three prohibited combinations: `--hdt-strategy single` with space-optimized storage, COTTAS, or `hdt,cottas`. Gzip and Brotli were exercised in both compression roles, but their ratios and throughput were not compared.

The `tsv`, `compress`, `decompress`, HDT-index, COTTAS-index, conversion-only, and validation-only modes each ran independently with exit code 0. A feasibility sweep crossed requested 8, 16, and 31 GB memory ceilings with three configurations on one million records. All nine cells completed at 1.34–1.93 GB resident memory, but the harness could not confirm that Docker limits were forwarded to all container invocations. The result therefore shows low measured memory for those runs, not survival under verified enforced limits.

Structured INFO increased the structural-variant graph from 89.01M to 96.75M triples (8.7%), N-Triples bytes by 7.4%, and HDT bytes by 7.8%. On 100,000 HG005 records, the increase was from 12.52M to 17.10M triples (36.6%). Raw INFO still retains alleles needed by expanded sample calls after the dependency correction in Section S8. Structured header output added only 0.03%: 17,098,746 versus 17,094,293 triples.

The cross-encoding experiment used a 10,000-line, 9,986-record single-sample fixture in both sample profiles and storage modes, with N-Triples, HDT, and COTTAS artifacts. All sixteen validations passed the fourteen preflight checks and Q1–Q13 under all four engines, on graphs of

0.96M triples expanded and 0.79M condensed. The complete condensed cell took 70.4 min. Two additional negative tests correctly refused single-HDT strategy with COTTAS or space-optimized storage, returning exit status 2.

References

- [1] Nobutaka Mitsuhashi, Licht Toyo-oka, Toshiaki Katayama, Minae Kawashima, Shuichi Kawashima, Kazunori Miyazaki, and Toshihisa Takagi. TogoVar: A comprehensive Japanese genetic variation database. *Human Genome Variation*, 9(1):44, December 2022.
- [2] Pierre Lindenbaum. Jvarkit: java-based utilities for Bioinformatics. figshare, 5 2015.
- [3] Jasmin K. Böhmer and Roel Janssen. Linked data in practice: An RDF-based approach with SPARQLing Genomics. Presentation with the SPARQLing Genomics user manual (v0.99.11), June 2020. Zenodo record, version 4.
- [4] Roel Janssen. SPARQLing Genomics. Open-source software, 2019. Accessed 2026-09-03.
- [5] Emanuel Diego S. Penha, Egiebade Iriabho, Alex Dussaq, Diana Magalhães de Oliveira, and Jonas S. Almeida. Isomorphic semantic mapping of variant call format (VCF2RDF). *Bioinformatics*, 33(4):547–548, 2017.
- [6] Toshiaki Katayama, Shuichi Kawashima, Gos Micklem, Shin Kawano, Jin-Dong Kim, Simon Kocbek, Shinobu Okamoto, Yue Wang, Hongyan Wu, Atsuko Yamaguchi, et al. Biohackathon series in 2013 and 2014: improvements of semantic interoperability in life science data and services. *F1000Research*, 8:1677, 2019.
- [7] Francesco Strozzi. bioruby-vcf2rdf: simple converter from VCF format to RDF Turtle format. GitHub repository, BioHackathon 2013, 2013. Accessed 2026-10-07.
- [8] Toshiaki Katayama, Mark D. Wilkinson, Kiyoko F. Aoki-Kinoshita, Shuichi Kawashima, Yasunori Yamamoto, Atsuko Yamaguchi, Shinobu Okamoto, Shin Kawano, Jin-Dong Kim, Yue Wang, et al. BioHackathon series in 2011 and 2012: penetration of ontology and linked data in life science domains. *Journal of Biomedical Semantics*, 5:5, 2014.
- [9] Joachim Baran. BioInterchange: genomic linked data converter for GFF, GVF, and VCF files. GitHub repository BioInterchange/BioInterchangeC, version 2.0.5 (commit 49d5701), MIT License, 2021. Accessed 2026-10-08.
- [10] Joachim Baran, Bibi Sehnaaz Begum Durgahee, Karen Eilbeck, Erick Antezana, Robert Hoehndorf, and Michel Dumontier. GFVO: the Genomic Feature and Variation Ontology. *PeerJ*, 3:e933, 2015.
- [11] Erik Garrison, Zev N. Kronenberg, Eric T. Dawson, Brent S. Pedersen, and Pjotr Prins. A spectrum of free software tools for processing the VCF variant call format: vcflib, bio-vcf, cyvcf2, hts-nim and slivar. *PLOS Computational Biology*, 18(5):e1009123, 2022.
- [12] Arto Bendiken, Raoul J.P. Bonnal, and Francesco Strozzi. ruby-rdf/rdf-vcf: Rdf vcf parser, September 2015.
- [13] Shivika Prasanna, Ajay Kumar, Deepthi Rao, Eduardo J. Simoes, and Praveen Rao. A scalable tool for analyzing genomic variants of humans using knowledge graphs and graph machine learning. *Frontiers in Big Data*, 7:1466391, 2025.
- [14] Pierre Larmande, Bertrand Pittolat, Ndomassi Tando, Yann Pomie, Bill Gates Happi Happi, Valentin Guignon, and Manuel Ruiz. AgroLD: a knowledge graph for the plant sciences. *BMC Genomic Data*, 26(Suppl 1):73, 2025.

- [15] Alexandrina Bodrug-Schepers, Hugo Chabane, Gabriela Montoya, Patricia Serrano-Alvarado, Richard Redon, and Alban Gaignard. Semantic Beacons: a framework to support federated querying over genomic variants and public Knowledge Graphs. In *Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, volume 4196 of *CEUR Workshop Proceedings*, Barcelona, Spain, February 2025. CEUR Workshop Proceedings.
- [16] Elias Crum. VCF Core Vocabulary. Version 2.1.3. GitHub repository, 2026. Version 2.1.3 with the SSSOM alignment set; source snapshot of 22 September 2026.
- [17] Jerven T. Bolleman, Christopher J. Mungall, Francesco Strozzi, Joachim Baran, Michel Dumontier, Raoul J. P. Bonnal, Robert Buels, Robert Hoehndorf, Takatomo Fujisawa, Toshiaki Katayama, and Peter J. A. Cock. FALDO: a semantic standard for describing the location of nucleotide and protein feature annotation. *Journal of Biomedical Semantics*, 7(1):39, December 2016.
- [18] Karen Eilbeck, Suzanna E Lewis, Christopher J Mungall, Mark Yandell, Lincoln Stein, Richard Durbin, and Michael Ashburner. The Sequence Ontology: a tool for the unification of genome annotations. *Genome Biology*, 6(5):R44, April 2005.
- [19] Laura Menotti, Mirco Cazzaro, Manuel Rueda, Ivo G Gut, and Gianmaria Silvello. HERO-Genomics: Bridging Genomic Data and Ontological Modelling. In *Semantic Web Applications and Tools for Health Care and Life Sciences (SWAT4HCLS 2025)*, Barcelona, Spain, February 2025. CEUR Workshop Proceedings.
- [20] Alex H. Wagner, Reece Hart, Larry Babb, Kori Kuzma, Helen Schuilenburg, James Stevenson, Terry O'Neill, Mrinal-Thomas Epic, Brad Holmes, Eric Moyer, Michael Baudis, Peter Robinson, Melissa Cline, Ronak Y. Patel, and Robert R. Freimuth. ga4gh/vrs 2.0.1, 2025.
- [21] Ben De Meester, Pieter Heyvaert, and Thomas Delva. RDF Mapping Language (RML), June 2024.
- [22] Sitt Min Oo, Gerald Haesendonck, Ben De Meester, and Anastasia Dimou. RMLStreamer-SISO: An RDF Stream Generator from Streaming Heterogeneous Data. In Ulrike Sattler, Aidan Hogan, Maria Keet, Valentina Presutti, João Paulo A. Almeida, Hideaki Takeda, Pierre Monnin, Giuseppe Pirrò, and Claudia d'Amato, editors, *The Semantic Web – ISWC 2022*, volume 13489, pages 697–713. Springer International Publishing, Cham, 2022. Series Title: Lecture Notes in Computer Science.
- [23] RDF/HDT contributors. hdt-cpp: HDT C++ library and tools, 2026. Built from the repository head at image build time; the exact revision is fixed by the VCF-RDFizer v3.1.0 image digest.
- [24] Javier D. Fernández, Miguel A. Martínez-Prieto, Claudio Gutiérrez, Axel Polleres, and Mario Arias. Binary rdf representation for publication and exchange (hdt). *Web Semantics: Science, Services and Agents on the World Wide Web*, 19:22–41, 2013.
- [25] FRINK OKN. hdtc: HDT creator, 2026. Version 1.1.0, computer software, MIT License.
- [26] Dennis Diefenbach and José M. Giménez-García. HDTCat: Let’s make HDT generation scale. In *The Semantic Web – ISWC 2020*, volume 12507 of *Lecture Notes in Computer Science*, pages 18–33, Cham, 2020. Springer.
- [27] Julián Arenas-Guerrero. pycottas: Python cottas library for compressing and querying rdf, 2026. RDFLib store backend, triple-pattern interface, and COTTAS conversion utilities.
- [28] Elias Crum. Comunica SPARQL COTTAS: a Comunica engine for query evaluation over COTTAS files, 2026. Version 0.1.0; npm package `@elias.crum/query-sparql-cottas`.

- [29] Petr Danecek, Adam Auton, Goncalo Abecasis, Cornelis A. Albers, Eric Banks, Mark A. DePristo, Robert E. Handsaker, Gerton Lunter, Gabor T. Marth, Stephen T. Sherry, Gilean McVean, Richard Durbin, and 1000 Genomes Project Analysis Group. The variant call format and VCFtools. *Bioinformatics*, 27(15):2156–2158, August 2011.
- [30] Petr Danecek, James K Bonfield, Jennifer Liddle, John Marshall, Valeriu Ohan, Martin O Pollard, Andrew Whitwham, Thomas Keane, Shane A McCarthy, Robert M Davies, and Heng Li. Twelve years of samtools and bcftools. *GigaScience*, 10(2):giab008, 2021.
- [31] Brent S. Pedersen and Aaron R. Quinlan. cyvcf2: fast, flexible variant analysis with Python. *Bioinformatics*, 33(12):1867–1869, 2017.
- [32] Hannah Bast and Björn Buchhold. QLever: A query engine for efficient SPARQL+Text search. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM)*, pages 647–656, 2017.
- [33] Ruben Taelman, Joachim Van Herwegen, Miel Vander Sande, and Ruben Verborgh. Comunica: A modular SPARQL query engine for the web. In *Proceedings of the 17th International Semantic Web Conference*, pages 239–255, Cham, 10 2018. Springer International Publishing.
- [34] Comunica Association. Querying local files from the command line, 2026.
- [35] Comunica Association. HDT: Querying HDT files with comunica, 2026.
- [36] J. Bradley Holmes, Eric Moyer, Lon Phan, Donna Maglott, and Brandi Kattman. SPDI: data model for variants and applications at NCBI. *Bioinformatics*, 36(6):1902–1907, 2020.
- [37] Jiwen Xin, Adam Mark, Cyrus Afrasiabi, Ginger Tsueng, et al. High-performance web services for querying gene and variant annotation. *Genome Biology*, 17:91, 2016.
- [38] Renato Iannella and Serena Villata. ODRL Information Model 2.2, February 2018.
- [39] Jonathan Lawson, Moran N. Cabili, Giselle Kerry, et al. The Data Use Ontology to streamline responsible access to human biomedical datasets. *Cell Genomics*, 1(2):100028, 2021.
- [40] David T. Miller, Kristy Lee, Noura S. Abul-Husn, Laura M. Amendola, et al. ACMG SF v3.2 list for reporting of secondary findings in clinical exome and genome sequencing: A policy statement of the American College of Medical Genetics and Genomics (ACMG). *Genetics in Medicine*, 25(8):100866, 2023.
- [41] Personal Genome Project, 2005.
- [42] Justin Wagner, Nathan D. Olson, Lindsay Harris, Ziad Khan, Jesse Farek, Medhat Mahmoud, Ana Stankovic, Vladimir Kovacevic, Byunggil Yoo, Neil Miller, et al. Benchmarking challenging small variants with linked and long reads. *Cell Genomics*, 2(5):100128, 2022.
- [43] Marta Byrska-Bishop, Uday S. Evani, Xuefang Zhao, et al. High-coverage whole-genome sequencing of the expanded 1000 Genomes Project cohort including 602 trios. *Cell*, 185(18):3426–3440.e19, 2022.
- [44] National Center for Biotechnology Information. ClinVar VCF Release (GRCh38), September 2026. Header declares `##fileformat=VCFv4.1`; accessed 8 September 2026.
- [45] Rafael C. Jiménez, Mateusz Kuzak, Monther Alhamdoosh, Michelle Barker, Bérénice Batut, Marc Borg, Salvador Capella-Gutiérrez, Neil P. Chue Hong, Martin Cook, Manuel Corpas, Michael Flannery, Leyla Garcia, Josep L. Gelpí, Simon Gladman, Carole Goble, Miguel González Ferreira, Alejandra Gonzalez-Beltran, Philip C. Griffin, Björn Grüning, Jon Hagberg, Petr Holub, Rob Hooft, Jon Ison, Sarah Jones, Florian Kaiser, Sunghwan Kim, Matthew

Krafczyk, Anna Krystalli, Hilmar Lapp, Ann Lonie, Céline Marek, Michael P. Mazanetz, Aleksandra Nenadic, Wolfgang Petersen, Esther Plomp, Mikhail Pomazkov, Karthik Ram, Stian Soiland-Reyes, Alexey Sokolov, Andreas Struck, Radoslaw Sucheki, Rutger Vos, Alan R. Williams, Patrick Brack, Marta W. Muelas, Alexander D. Diehl, Jean-Baptiste Poline, Teresa K. Atwood, Alexander Mansurov, Larisa N. Soldatova, Carl Kingsford, Paul J. Kersey, Mark D. McDowall, Magne Østeras, Cristina Guijarro-Clarke, and Steve Crouch. Four simple recommendations to encourage best practices in research software. *F1000Research*, 6:876, 2017.